

✓ PolyFlow — Complete Pipeline

Single notebook combining both tasks:

Section	Task	Description
Parts 1–4	Shared	Feature extraction (Skeleton, Polyline, X3D-XS) + Zero-fix
Part 5–7 (A)	Anomaly Detection	Binary classification: Normal vs Anomalous
Part 5–7 (B)	Event Recognition	14-class classification: identify the anomaly type

Run Parts 1–4 once. Then run either or both of the task sections.

✓ Setup

```
# — Cell S1: Install —————
!pip install -q ultralytics pytorchvideo scikit-learn matplotlib seaborn opencv-p
import importlib
for pkg in ['torch','torchvision','sklearn','matplotlib','seaborn','cv2','numpy']:
    try:
        m=importlib.import_module(pkg)
        print(f'  ✓ {pkg} {getattr(m,"__version__","?")}')
    except ImportError:
        print(f'  ✗ {pkg}')
```

```

132.7/132.7 kB 7.6 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
50.2/50.2 kB 4.9 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
42.2/42.2 kB 4.1 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
1.2/1.2 MB 40.9 MB/s eta 0:00:00
36.3/36.3 MB 32.0 MB/s eta 0:00:00
Building wheel for pytorchvideo (setup.py) ... done
Building wheel for fvcore (setup.py) ... done
Building wheel for iopath (setup.py) ... done
✓ torch 2.10.0+cu128
✓ torchvision 0.25.0+cu128
✓ sklearn 1.6.1
✓ matplotlib 3.10.0
✓ seaborn 0.13.2
✓ cv2 4.13.0
✓ numpy 2.0.2
```

```
# — Cell S2: Mount Drive & GPU check —————
import torch
if torch.cuda.is_available():
    g=torch.cuda.get_device_properties(0)
```

```

print(f'✅ GPU: {torch.cuda.get_device_name(0)} VRAM: {g.total_memory/1e9:.1f} GB')
else:
    print('⚠️ No GPU – switch Runtime → T4 GPU')

```

✅ GPU: Tesla T4 VRAM: 15.6 GB

```

# — Cell S3: Dataset path reference —————
#
# | ANOMALY DETECTION TASK – SPLIT FILE PATHS |
# |-----|
# | Training list: |
# | .../Anomaly-Detection-Dataset_v2/Anomaly_Train.txt |
# |   → Lists ~810 training videos (anomaly + normal mixed) |
# |   → Format: <VideoName>.mp4 (one per line, no label column) |
# | |
# | Test list: |
# | .../UCF_Crimes-Train-Test-Split/Anomaly_Detection_splits/Anomaly_Test.txt |
# |   → Lists ~140 test videos |
# |   → Same format: <VideoName>.mp4 |
# | |
# | Temporal annotations (frame-level GT for test videos): |
# | .../Anomaly-Detection-Dataset_v2/Temporal_Anomaly_Annotation_for_Testing_Videos |
# |   → Format: VideoName.mp4 EventType start1 end1 start2 end2 |
# |   → -1 means no second instance |
# |   → Used to compute frame-level AUC (primary metric for this task) |
# |-----|
import os
from pathlib import Path

BASE = '/content/drive/MyDrive/Dataset/Anomaly-Detection-Dataset_v2'

PATHS = {
    'dataset_root'      : BASE,
    'train_list'        : f'{BASE}/Anomaly_Train.txt',
    'test_list'         : f'{BASE}/UCF_Crimes-Train-Test-Split/Anomaly_Detection_splits/Anomaly_Test.txt',
    'temporal_annot'    : f'{BASE}/Temporal_Anomaly_Annotation_for_Testing_Videos.txt',
    'skel_features'     : '/content/drive/MyDrive/Dataset/skeleton_features_v3',
    'poly_features'     : '/content/drive/MyDrive/Dataset/polyline_features_v1',
    'x3d_features'      : '/content/drive/MyDrive/Dataset/x3d_features_v1',
    'log_dir'           : '/content/drive/MyDrive/PolyFlow_AnomalyDetection_Logs',
    'class_ids'         : f'{BASE}/UCF_Crimes-Train-Test-Split/Action_Regnition_split/class_ids.txt',
    'split_dir'         : f'{BASE}/UCF_Crimes-Train-Test-Split/Action_Regnition_split'
}

print('Path check:')
for k,v in PATHS.items():
    exists = os.path.exists(v)
    icon = '✅' if exists else '❌ MISSING'
    print(f' {icon} {k}: {v}')

os.makedirs(PATHS['log_dir'], exist_ok=True)

```

Path check:

- ✓ dataset_root: /content/drive/MyDrive/Dataset/Anomaly-Detection-Dataset_v2
- ✓ train_list: /content/drive/MyDrive/Dataset/Anomaly-Detection-Dataset_v2/Anom
- ✓ test_list: /content/drive/MyDrive/Dataset/Anomaly-Detection-Dataset_v2/UCF_C
- ✓ temporal_annot: /content/drive/MyDrive/Dataset/Anomaly-Detection-Dataset_v2/
- ✓ skel_features: /content/drive/MyDrive/Dataset/skeleton_features_v3
- ✓ poly_features: /content/drive/MyDrive/Dataset/polyline_features_v1
- ✓ x3d_features: /content/drive/MyDrive/Dataset/x3d_features_v1
- ✓ log_dir: /content/drive/MyDrive/PolyFlow_AnomalyDetection_Logs
- ✓ class_ids: /content/drive/MyDrive/Dataset/Anomaly-Detection-Dataset_v2/UCF_C
- ✓ split_dir: /content/drive/MyDrive/Dataset/Anomaly-Detection-Dataset_v2/UCF_C

Part 1 — Skeleton Feature Extraction (YOLO11s-pose → 96-dim)

🕒 ~2-4 hrs on T4. Skips existing files automatically.

— Cell 1.1: Config

```
SKEL_CFG = {
    'dataset_root' : PATHS['dataset_root'],
    'save_root'    : PATHS['skel_features'],
    'yolo_model'   : 'yolo11s-pose.pt',
    'frame_interval': 16,
    'conf'         : 0.25,
    'batch_size'   : 32,
    'imgsz'        : 640,
    'overwrite'    : False,
}
# All video subfolders under dataset_root that contain raw videos
ALL_SUBSETS = [
    'Anomaly-Videos-Part-1', 'Anomaly-Videos-Part-2',
    'Anomaly-Videos-Part-3', 'Anomaly-Videos-Part-4',
    'Training-Normal-Videos-Part-1', 'Training-Normal-Videos-Part-2',
    'Testing_Normal_Videos',
]
print('Skeleton config:'); [print(f'  {k}: {v}') for k,v in SKEL_CFG.items()]
```

— Cell 1.2: Skeleton library

```
import time, cv2, numpy as np, torch
from ultralytics import YOLO

LIMB_PAIRS = [(0,1),(0,2),(1,3),(2,4),(5,6),(5,7),(7,9),(6,8),(8,10),
              (5,11),(6,12),(11,12),(11,13),(13,15),(12,14),(14,16)]
ANGLE_TRIPLETS = [(5,7,9),(6,8,10),(7,5,11),(8,6,12),(5,11,13),
                  (6,12,14),(11,13,15),(12,14,16),(0,5,6),(11,12,0)]
DIM_LIMB,DIM_CONF,DIM_ANGLE,DIM_BBOX,DIM_COUNT = 32,17,10,4,1
DIM_STATIC = DIM_LIMB+DIM_CONF+DIM_ANGLE+DIM_BBOX+DIM_COUNT # 64
```

```

SKEL_DIM    = DIM_STATIC + DIM_LIMB  # 96
VIDEO_EXTS  = {'.mp4', '.avi', '.mov', '.mkv', '.mpg', '.mpeg'}

def setup_device():
    dev='cuda' if torch.cuda.is_available() else 'cpu'
    half=dev=='cuda'
    print(f'[GPU] {torch.cuda.get_device_name(0)} fp16={half}' if dev=='cuda' else
    return dev, half

def _angle(a,b,c):
    ba,bc=a-b,c-b; n1,n2=np.linalg.norm(ba),np.linalg.norm(bc)
    if n1<1e-6 or n2<1e-6: return 0.5
    return float(np.arccos(np.clip(np.dot(ba,bc)/(n1*n2), -1.,1.))/np.pi)

def _kpts_to_vec(kpts_np,box_xyxy,fh,fw,num_persons):
    W=float(box_xyxy[2]-box_xyxy[0]); H=float(box_xyxy[3]-box_xyxy[1])
    if W<1. or H<1.: return np.zeros(DIM_STATIC,dtype=np.float32)
    norm=np.zeros((17,2),dtype=np.float32)
    for i in range(17):
        norm[i,0]=(kpts_np[i,0]-box_xyxy[0])/W; norm[i,1]=(kpts_np[i,1]-box_xyxy[
    vec=np.zeros(DIM_STATIC,dtype=np.float32)
    for idx,(i,j) in enumerate(LIMB_PAIRS):
        vec[idx*2]=norm[j,0]-norm[i,0]; vec[idx*2+1]=norm[j,1]-norm[i,1]
    nv=np.linalg.norm(vec[:DIM_LIMB])
    if nv>1e-6: vec[:DIM_LIMB]/=nv
    off=DIM_LIMB
    for i in range(17): vec[off+i]=kpts_np[i,2] if kpts_np.shape[1]>2 else 0.5
    off+=DIM_CONF
    for ai,(i,j,k) in enumerate(ANGLE_TRIPLETS): vec[off+ai]=_angle(norm[i],norm[
    off+=DIM_ANGLE
    vec[off+0]=(box_xyxy[0]+box_xyxy[2])/2./max(fw,1.); vec[off+1]=(box_xyxy[1]+b
    vec[off+2]=W/max(fw,1.); vec[off+3]=H/max(fh,1.)
    off+=DIM_BBOX; vec[off]=min(num_persons,5)/5.
    return vec

def batch_to_skel(frames,model,conf,dev,half,imgsz):
    if not frames: return []
    results=model(frames,verbose=False,conf=conf,device=dev,half=half,imgsz=imgsz
    out=[]
    for idx,res in enumerate(results):
        fh,fw=frames[idx].shape[:2]
        if res.keypoints is None or len(res.keypoints.data)==0 or res.bboxes is No
            out.append(np.zeros(DIM_STATIC,dtype=np.float32)); continue
        boxes=res.bboxes.xyxy.cpu().numpy()
        areas=(boxes[:,2]-boxes[:,0])*(boxes[:,3]-boxes[:,1])
        best=int(np.argmax(areas))
        out.append(_kpts_to_vec(res.keypoints.data[best].cpu().numpy(),boxes[best
    return out

def read_sampled_frames(vp,interval):
    cap=cv2.VideoCapture(str(vp))
    if not cap.isOpened(): return None

```

```

frames,fi=[],0
while True:
    ret,f=cap.read()
    if not ret: break
    if fi%interval==0: frames.append(f)
    fi+=1
cap.release(); return frames if frames else None

def extract_skeleton(vp,model,cfg,dev,half):
    frames=read_sampled_frames(vp,cfg['frame_interval'])
    if frames is None: return None
    static=[]
    for s in range(0,len(frames),cfg['batch_size']):
        static.extend(batch_to_skel(frames[s:s+cfg['batch_size']],model,cfg['conf'])
    if not static: return None
    st=np.stack(static,axis=0); limb=st[:, :DIM_LIMB]
    vel=np.zeros_like(limb); vel[1:]=limb[1:]-limb[:-1]
    vel/=np.linalg.norm(vel,axis=1,keepdims=True).clip(min=1e-6)
    return np.concatenate([st,vel],axis=1)

print(f'Skeleton library loaded. Feature dim = {SKEL_DIM}')

```

```

# — Cell 1.3: Run extraction —————
dev,half=setup_device()
yolo_model=YOLO(SKEL_CFG['yolo_model'])
ds_root=Path(SKEL_CFG['dataset_root']); sk_save=Path(SKEL_CFG['save_root'])
found=[s for s in ALL_SUBSETS if (ds_root/s).is_dir()]
grand_done=grand_skip=grand_fail=0
for subset in found:
    vid_root=ds_root/subset; out_root=sk_save/subset
    vids=sorted([p for p in vid_root.rglob('*') if p.suffix.lower() in VIDEO_EXTS])
    done=skip=fail=0
    for vp in vids:
        rel=vp.relative_to(vid_root); sp=out_root/rel.with_suffix('.txt')
        if sp.exists() and not SKEL_CFG['overwrite']: skip+=1; continue
        feats=extract_skeleton(vp,yolo_model,SKEL_CFG,dev,half)
        if feats is None: fail+=1; continue
        sp.parent.mkdir(parents=True,exist_ok=True)
        np.savetxt(str(sp),feats,fmt='%.6f'); done+=1
    grand_done+=done; grand_skip+=skip; grand_fail+=fail
    print(f'{subset}: saved={done} skipped={skip} failed={fail}')
print(f'\nSkeleton DONE — Extracted:{grand_done} Skipped:{grand_skip} Failed:{gra

```

Part 2 — Polyline Feature Extraction (LK Optical Flow → 64-dim)

⌚ ~1–2 hrs on T4. CPU-only (no GPU needed).

```
# — Cell 2.1: Config
POLY_CFG = {
    'dataset_root' : PATHS['dataset_root'],
    'save_root'     : PATHS['poly_features'],
    'frame_interval': 16,
    'max_corners'   : 80,
    'quality_level' : 0.01,
    'min_distance'  : 10,
    'poly_dim'      : 64,
    'overwrite'     : False,
}
POLY_DIM = POLY_CFG['poly_dim']
LK_PARAMS = dict(winSize=(15,15), maxLevel=2, criteria=(cv2.TERM_CRITERIA_EPS|cv2.TERM_CRITERIA_MAX_ITER), minDistance=10)
FEATURE_PARAMS = dict(maxCorners = POLY_CFG['max_corners'], qualityLevel = POLY_CFG['quality_level'], minDistance = 10)
print(f'Polyline config ready. dim={POLY_DIM}')
```

```
# — Cell 2.2: Polyline library
def _encode_polylines(trajectories, frame_h, frame_w):
    feat = np.zeros(POLY_DIM, dtype=np.float32)
    if not trajectories: return feat
    H, W = max(frame_h, 1), max(frame_w, 1)
    all_speeds, all_dirs, all_curves, net_disps, loiter_ratios = [], [], [], [], []
    spatial_counts = np.zeros((2, 4), dtype=np.float32)
    for traj in trajectories:
        if len(traj) < 2: continue
        traj = np.array(traj, dtype=np.float32); diffs = np.diff(traj, axis=0)
        speeds = np.linalg.norm(diffs, axis=1); all_speeds.extend(speeds.tolist())
        angles = np.arctan2(diffs[:, 1], diffs[:, 0]); all_dirs.extend(angles.tolist())
        if len(diffs) > 1:
            ad = np.diff(angles); ad = (ad + np.pi) % (2 * np.pi) - np.pi
            all_curves.extend(np.abs(ad).tolist())
        net = traj[-1] - traj[0]; net_disps.append(np.array([net[0]/W, net[1]/H]))
        mid = traj[len(traj)//2]
        gy = int(np.clip(mid[1]/H*2, 0, 1)); gx = int(np.clip(mid[0]/W*4, 0, 3))
        spatial_counts[gy, gx] += 1
        path_len = speeds.sum() + 1e-6; net_dist = np.linalg.norm(net) + 1e-6
        loiter_ratios.append(1.0 - min(net_dist/path_len, 1.0))
    if all_speeds:
        h, _ = np.histogram(all_speeds, bins=16, range=(0, 50))
        feat[0:16] = h.astype(np.float32) / (sum(all_speeds) + 1e-6)
    if all_dirs:
        h, _ = np.histogram(all_dirs, bins=8, range=(-np.pi, np.pi))
        feat[16:24] = h.astype(np.float32) / (len(all_dirs) + 1e-6)
    if all_curves:
        h, _ = np.histogram(all_curves, bins=8, range=(0, np.pi))
        feat[24:32] = h.astype(np.float32) / (len(all_curves) + 1e-6)
    if net_disps:
        nd = np.array(net_disps)
        feat[32] = float(nd[:, 0].mean()); feat[33] = float(nd[:, 1].mean())
        feat[34] = float(nd[:, 0].std()); feat[35] = float(nd[:, 1].std())
        feat[36] = float(nd[:, 0].max()); feat[37] = float(nd[:, 1].max())
```

```

        feat[38]=float(np.abs(nd[:,0]).mean()); feat[39]=float(np.abs(nd[:,1]).me
feat[40:48]=(spatial_counts/(spatial_counts.sum()+1e-6)).flatten()
if all_speeds:
    n=len(all_speeds); bs=max(n//8,1)
    for b in range(8):
        sl=all_speeds[b*bs:(b+1)*bs] if b<7 else all_speeds[b*bs:]
        feat[48+b]=float(np.mean(sl)) if sl else 0.
n_tracks=len(trjectories)
feat[56]=float(n_tracks)/max(POLY_CFG['max_corners'],1)
feat[57]=float(np.mean([len(t) for t in trajectories]))/max(POLY_CFG['frame_i
feat[58]=float(np.std([len(t) for t in trajectories])) if trajectories else 0
h_vals=[len(t)/max(POLY_CFG['frame_interval'],1) for t in trajectories]
if h_vals:
    h_arr=np.array(h_vals).clip(1e-9,1.)
    feat[59]=float(-np.sum(h_arr*np.log(h_arr))/np.log(len(h_arr)+2))
if loiter_ratios:
    lr=np.array(loiter_ratios)
    feat[60]=float(lr.mean()); feat[61]=float(lr.max())
    feat[62]=float(lr.std()); feat[63]=float((lr>0.7).mean())
return feat

def extract_polylines(video_path,cfg):
    cap=cv2.VideoCapture(str(video_path))
    if not cap.isOpened(): return None
    segment_features=[]
    ret,prev_frame=cap.read()
    if not ret: cap.release(); return None
    prev_gray=cv2.cvtColor(prev_frame,cv2.COLOR_BGR2GRAY)
    fh,fw=prev_gray.shape
    p0=cv2.goodFeaturesToTrack(prev_gray,mask=None,**FEATURE_PARAMS)
    if p0 is None: p0=np.zeros((0,1,2),dtype=np.float32)
    segment_trajs=[[([pt[0,0],pt[0,1]]) for pt in p0]
    frame_idx=1
    while True:
        ret,frame=cap.read()
        if not ret: break
        gray=cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY)
        if len(p0)>0:
            p1,st,_=cv2.calcOpticalFlowPyrLK(prev_gray,gray,p0,None,**LK_PARAMS)
            if p1 is not None:
                good_new=p1[st==1]; good_idx=np.where(st.flatten()==1)[0]
                for ni,oi in enumerate(good_idx):
                    if oi<len(segment_trajs):
                        segment_trajs[oi].append((float(good_new[ni,0,0]),float(g
                        p0=good_new.reshape(-1,1,2)
                    else: p0=np.zeros((0,1,2),dtype=np.float32)
        if frame_idx>0 and frame_idx%cfg['frame_interval']==0:
            segment_features.append(_encode_polylines(segment_trajs,fh,fw))
            p0=cv2.goodFeaturesToTrack(gray,mask=None,**FEATURE_PARAMS)
            if p0 is None: p0=np.zeros((0,1,2),dtype=np.float32)
            segment_trajs=[[([pt[0,0],pt[0,1]]) for pt in p0]
            prev_gray=gray; frame_idx+=1

```

```

    if segment_trajs and any(len(t)>1 for t in segment_trajs):
        segment_features.append(_encode_polylines(segment_trajs,fh,fw))
    cap.release()
    return np.stack(segment_features,axis=0) if segment_features else None

print(f'Polyline library loaded. dim={POLY_DIM}')

```

```

# — Cell 2.3: Run polyline extraction —————
ds_root=Path(POLY_CFG['dataset_root']); poly_save=Path(POLY_CFG['save_root'])
found=[s for s in ALL_SUBSETS if (ds_root/s).is_dir()]
grand_done=grand_skip=grand_fail=0
for subset in found:
    vid_root=ds_root/subset; out_root=poly_save/subset
    vids=sorted([p for p in vid_root.rglob('*') if p.suffix.lower() in VIDEO_EXTS])
    done=skip=fail=0
    for vp in vids:
        rel=vp.relative_to(vid_root); sp=out_root/rel.with_suffix('.txt')
        if sp.exists() and not POLY_CFG['overwrite']: skip+=1; continue
        feats=extract_polylines(vp,POLY_CFG)
        if feats is None: fail+=1; continue
        sp.parent.mkdir(parents=True,exist_ok=True)
        np.savetxt(str(sp),feats,fmt='%.6f'); done+=1
    grand_done+=done; grand_skip+=skip; grand_fail+=fail
    print(f'{subset}: saved={done} skipped={skip} failed={fail}')
print(f'\nPolyline DONE – Extracted:{grand_done} Skipped:{grand_skip} Failed:{gra

```

✓ Part 3 — X3D-XS Visual Feature Extraction (192-dim)

🕒 ~3–6 hrs on T4.

```

# — Cell 3.1: Config & model —————
import torch, torch.nn as nn
from pytorchvideo.models.hub import x3d_xs

X3D_CFG = {
    'dataset_root' : PATHS['dataset_root'],
    'save_root'    : PATHS['x3d_features'],
    'frame_interval': 16, 'clip_frames': 4,
    'crop_size': 160,   'batch_clips': 8,
    'overwrite': False,
}

device='cuda' if torch.cuda.is_available() else 'cpu'

class X3DExtractor(nn.Module):
    def __init__(self,base):
        super().__init__()
        self.stages=nn.ModuleList(list(base.blocks[:-1]))

```

```

        self.pool=nn.AdaptiveAvgPool3d(1)
    def forward(self,x):
        for s in self.stages: x=s(x)
        return self.pool(x).flatten(1)

_base=x3d_xs(pretrained=True); _base.eval()
x3d_extractor=X3DExtractor(_base).eval().to(device)
MEAN_X3D=np.array([0.45,0.45,0.45],dtype=np.float32)
STD_X3D =np.array([0.225,0.225,0.225],dtype=np.float32)
with torch.no_grad():
    dummy=torch.zeros(1,3,X3D_CFG['clip_frames'],X3D_CFG['crop_size'],X3D_CFG['cr
    X3D_DIM=x3d_extractor(dummy).shape[1]
assert X3D_DIM==192, f'Expected 192, got {X3D_DIM}'
print(f'X3D-XS ready. dim={X3D_DIM} ✅')

```

— Cell 3.2: Run X3D extraction —————

```

def preprocess_x3d(frame,cs):
    frame=cv2.cvtColor(frame,cv2.COLOR_BGR2RGB)
    h,w=frame.shape[:2]; side=min(h,w)
    y0=(h-side)//2; x0=(w-side)//2
    frame=frame[y0:y0+side,x0:x0+side]
    frame=cv2.resize(frame,(cs,cs),interpolation=cv2.INTER_LINEAR)
    return (frame.astype(np.float32)/255.-MEAN_X3D)/STD_X3D

def extract_x3d(vp,cfg):
    cap=cv2.VideoCapture(str(vp))
    if not cap.isOpened(): return None
    fi=cfg['frame_interval']; cf=cfg['clip_frames']; cs=cfg['crop_size']; bc=cfg[
    sample_idx=set(np.linspace(0,fi-1,cf,dtype=int))
    all_feats=[]; clip_batch=[]; cur_seg=[]; fc=0
    while True:
        ret,frame=cap.read()
        if not ret: break
        if (fc%fi) in sample_idx: cur_seg.append(preprocess_x3d(frame,cs))
        fc+=1
        if fc%fi==0:
            if len(cur_seg)==cf: clip_batch.append(np.stack(cur_seg,axis=0))
            cur_seg=[]
            if len(clip_batch)==bc:
                t=torch.from_numpy(np.stack(clip_batch,0)).permute(0,4,1,2,3).to(
                with torch.no_grad(): all_feats.append(x3d_extractor(t).cpu().num
                clip_batch=[]
            if clip_batch:
                t=torch.from_numpy(np.stack(clip_batch,0)).permute(0,4,1,2,3).to(device)
                with torch.no_grad(): all_feats.append(x3d_extractor(t).cpu().numpy())
            cap.release()
    return np.concatenate(all_feats,0) if all_feats else None

ds_root=Path(X3D_CFG['dataset_root']); x3d_save=Path(X3D_CFG['save_root'])
found=[s for s in ALL_SUBSETS if (ds_root/s).is_dir()]
grand_done=grand_skip=grand_fail=0
for subset in found:

```

```

vid_root=ds_root/subset; out_root=x3d_save/subset
vids=sorted([p for p in vid_root.rglob('*') if p.suffix.lower() in VIDEO_EXTS])
done=skip=fail=0
for vp in vids:
    rel=vp.relative_to(vid_root); sp=out_root/rel.with_suffix('.txt')
    if sp.exists() and not X3D_CFG['overwrite']: skip+=1; continue
    feats=extract_x3d(vp,X3D_CFG)
    if feats is None: fail+=1; continue
    sp.parent.mkdir(parents=True,exist_ok=True)
    np.savetxt(str(sp),feats,fmt='%.6f'); done+=1
grand_done+=done; grand_skip+=skip; grand_fail+=fail
print(f'{subset}: saved={done} skipped={skip} failed={fail}')
print(f'\nX3D DONE – Extracted:{grand_done} Skipped:{grand_skip} Failed:{grand_fa

```

✓ Part 4 — Zero-Skeleton Fix

Some outdoor/dark scenes have no detectable persons. Retry at lower confidence, then mark remaining as visual-only.

```

# — Cell 4.1: Scan for all-zero skeleton files —————
skel_root=Path(PATHS['skel_features'])
zero_files=[]; total=0
for txt in sorted(skel_root.rglob('*.txt')):
    total+=1
    try:
        arr=np.loadtxt(str(txt),dtype=np.float32)
        if arr.ndim==1: arr=arr.reshape(1,-1)
        if not np.any(arr!=0): zero_files.append(txt)
    except: pass
print(f'Checked: {total} All-zero: {len(zero_files)} ({len(zero_files)/max(total

```

Checked: 1900 All-zero: 115 (6.1%)

```

# — Cell 4.2: Retry at lower conf; mark remaining as STILL_ZERO —————
VIDEO_EXTS = {'.mp4','.avi','.mov','.mkv','.mpg','.mpeg'}

def find_video(txt_path,dataset_root):
    stem=txt_path.stem
    # Added check to ensure dataset_root is accessible before iterating
    if not Path(dataset_root).is_dir():
        print(f" ❌ Error: Dataset root directory not found or inaccessible: {dataset_root}")
        return None
    for subset_dir in Path(dataset_root).iterdir():
        if not subset_dir.is_dir(): continue
        for vid in subset_dir.rglob('*'):
            if vid.stem==stem and vid.suffix.lower() in VIDEO_EXTS: return vid
    return None

```

```

# Guard: run Cell 4.1 first; if skipped, scan now
if 'zero_files' not in dir():
    print('⚠️ zero_files not found – running scan now (Cell 4.1 was skipped)...
    skel_root=Path(PATHS['skel_features'])
    zero_files=[]; total=0
    for txt in sorted(skel_root.rglob('*.txt')):
        total+=1
        try:
            arr=np.loadtxt(str(txt),dtype=np.float32)
            if arr.ndim==1: arr=arr.reshape(1,-1)
            if not np.any(arr!=0): zero_files.append(txt)
        except: pass
    print(f'Checked: {total} All-zero: {len(zero_files)} ({len(zero_files)/max(t

retry_pairs=[]
for txt in zero_files:
    vid=find_video(txt,PATHS['dataset_root'])
    if vid: retry_pairs.append((vid,txt))

still_zero=set(); improved=0
if retry_pairs:
    dev2='cuda' if torch.cuda.is_available() else 'cpu'
    retry_model=YOLO(SKEL_CFG['yolo_model'])
    for vid_path,txt_path in retry_pairs:
        cfg2=dict(SKEL_CFG); cfg2['conf']=0.15
        feats=extract_skeleton(vid_path,retry_model,cfg2,dev2,dev2=='cuda')
        if feats is not None and np.any(feats!=0):
            np.savetxt(str(txt_path),feats,fmt='%.6f'); improved+=1
        else:
            still_zero.add(txt_path.stem)
    print(f'Improved: {improved} Still-zero: {len(still_zero)}')
else:
    print('No zero-skeleton files found or dataset root inaccessible.')
STILL_ZERO_STEMS=still_zero
print(f'Visual-only mode will apply to {len(STILL_ZERO_STEMS)} videos.')

```

```

-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_2292/2671464576.py in <cell line: 0>()
    36 if retry_pairs:
    37     dev2='cuda' if torch.cuda.is_available() else 'cpu'
---> 38     retry_model=YOLO(SKEL_CFG['yolo_model'])
    39     for vid_path,txt_path in retry_pairs:
    40         cfg2=dict(SKEL_CFG); cfg2['conf']=0.15

NameError: name 'YOLO' is not defined

```

```
# Safety fallback if Part 4 was skipped
if 'STILL_ZERO_STEMS' not in dir():
    STILL_ZERO_STEMS = set()
    print('⚠️ STILL_ZERO_STEMS not found – defaulting to empty set (no zero-ske:

⚠️ STILL_ZERO_STEMS not found – defaulting to empty set (no zero-skeleton videos)
```

Task A — Anomaly Detection (Binary: Normal vs Anomalous)

Output: Anomaly score $\in [0, 1]$ for each video + frame-level AUC

Split files: `Anomaly_Train.txt` / `Anomaly_Test.txt`

⬆️ Parts 1–4 must be complete before running this section.

Part 5 — PolyGuided Fusion Training (Binary: Normal=0 vs Anomalous=1)

Split used: `Anomaly_Train.txt` / `Anomaly_Test.txt`

The train list contains both anomaly and normal videos (no labels in the file — we derive labels from the folder name).

- Videos from `Anomaly-Videos-Part-*` → label **1 (Anomalous)**
- Videos from `Training-Normal-Videos-*` or `Testing_Normal_Videos` → label **0 (Normal)**

```
# — Cell 5.1: Paths, seeds, binary config
import torch, os, json, math, random, time, shutil
import numpy as np, matplotlib.pyplot as plt, seaborn as sns
from pathlib import Path
from torch.utils.data import Dataset, DataLoader
from sklearn.metrics import f1_score, accuracy_score, roc_auc_score, roc_curve, c
```

```

import torch.nn as nn

SEED=42; random.seed(SEED); np.random.seed(SEED)
torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic=True; torch.backends.cudnn.benchmark=False

SKEL_ROOT = PATHS['skel_features']
POLY_ROOT = PATHS['poly_features']
X3D_ROOT  = PATHS['x3d_features']
LOG_DIR   = PATHS['log_dir']
DEVICE    = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

NUM_CLASSES = 2
BINARY_NAMES = ['Normal', 'Anomalous']

# Folders that contain anomaly videos
ANOMALY_SUBSETS = {'Anomaly-Videos-Part-1', 'Anomaly-Videos-Part-2',
                   'Anomaly-Videos-Part-3', 'Anomaly-Videos-Part-4'}

def path_to_binary_label(video_path_str):
    """Return 1 if the video is from an anomaly subset, else 0."""
    p=Path(video_path_str)
    for part in p.parts:
        if part in ANOMALY_SUBSETS: return 1
    return 0

print(f'Device: {DEVICE} Classes: {BINARY_NAMES}')

```

Device: cuda Classes: ['Normal', 'Anomalous']

```

# — Cell 5.2: Build feature indexes —————
def build_index(root):
    root=Path(root)
    if not root.exists(): print(f' ⚠ Missing: {root}'); return {}
    idx={}
    for p in root.rglob('*.txt'):
        if p.stem not in idx: idx[p.stem]=str(p)
    print(f' ✅ {root.name}: {len(idx)} files')
    return idx

print('Building indexes...')
SKEL_IDX=build_index(SKEL_ROOT)
POLY_IDX=build_index(POLY_ROOT)
X3D_IDX =build_index(X3D_ROOT)

```

```

Building indexes...
✅ skeleton_features_v3: 1900 files
✅ polyline_features_v1: 1900 files
✅ x3d_features_v1: 1900 files

```

```

# — Cell 5.3: Load Anomaly_Train.txt / Anomaly_Test.txt —————
#

```

```

# IMPORTANT: The Anomaly_Train.txt and Anomaly_Test.txt files list video
# file paths relative to the dataset root, one per line.
# Example line: Anomaly-Videos-Part-1/Abuse/Abuse001_x264.mp4
# OR just:      Abuse001_x264.mp4
# We handle both formats below.

def load_anomaly_split(path):
    """Load Anomaly_Train.txt or Anomaly_Test.txt.
    Returns list of (stem, binary_label) tuples.
    Label is inferred from path: anomaly subset folder → 1, else → 0.
    """
    p=Path(path)
    if not p.exists(): print(f' ✗ Not found: {p}'); return []
    entries=[]
    for line in p.read_text(errors='ignore').splitlines():
        line=line.strip()
        if not line or line.startswith('#'): continue
        # Drop label column if present (some versions have a 0/1 at end)
        parts=line.split()
        rel=parts[0] # e.g. 'Abuse/Abuse001_x264.mp4' or just 'Abuse001_x264.mp4'
        stem=Path(rel).stem
        # Infer label from path OR from provided label column
        if len(parts)>=2 and parts[-1] in ('0','1'):
            label=int(parts[-1])
        else:
            # Guess from stem prefix: Normal_Videos → 0, else → 1
            label=0 if stem.startswith('Normal') else 1
        entries.append((stem,label))
    n0=sum(1 for _,l in entries if l==0)
    n1=sum(1 for _,l in entries if l==1)
    print(f' {p.name}: {len(entries)} entries | Normal={n0} Anomalous={n1}')
    return entries

print('Loading splits...')
train_entries=load_anomaly_split(PATHS['train_list'])
test_entries =load_anomaly_split(PATHS['test_list'])

# Remove any overlap
train_stems={s for s,_ in train_entries}
test_entries=[(s,l) for s,l in test_entries if s not in train_stems]
print(f'After dedup → Train: {len(train_entries)} | Test: {len(test_entries)}')

```

```

Loading splits...
Anomaly_Train.txt: 1610 entries | Normal=800 Anomalous=810
Anomaly_Test.txt: 290 entries | Normal=150 Anomalous=140
After dedup → Train: 1610 | Test: 290

```

— Cell 5.4: Binary Dataset

```

def resample(feats,n):
    if feats.ndim==1: feats=feats.reshape(1,-1)
    T,D=feats.shape
    if T==0: return np.zeros((n,D),dtype=np.float32)

```

```

if T==n: return feat.astype(np.float32)
if T==1:
    r=np.repeat(feat,n,axis=0)
    return r+np.random.normal(0,1e-4,r.shape).astype(np.float32)
idx=np.linspace(0,T-1,n)
lo=np.floor(idx).astype(int).clip(0,T-1)
hi=np.ceil(idx).astype(int).clip(0,T-1)
alpha=(idx-lo)[:None]
return (feat[lo]*(1-alpha)+feat[hi]*alpha).astype(np.float32)

class AnomalyDetectionDataset(Dataset):
    """Binary dataset for Anomaly Detection task.
    entries = list of (stem, binary_label) from load_anomaly_split().
    """
    def __init__(self,entries,seg_num=32,split='?',zero_stems=None):
        self.seg_num=seg_num; self.zero_stems=zero_stems or set()
        valid=[]; miss_sk=miss_po=miss_x3=0
        for stem,label in entries:
            sk=SKEL_IDX.get(stem); po=POLY_IDX.get(stem); x3=X3D_IDX.get(stem)
            if sk is None: miss_sk+=1; continue
            if po is None: miss_po+=1
            if x3 is None: miss_x3+=1
            valid.append({'stem':stem,'label':label,'sk':sk,'po':po,'x3':x3})
        self.samples=valid
        n0=sum(1 for s in valid if s['label']==0)
        n1=sum(1 for s in valid if s['label']==1)
        print(f'[{split}] {len(valid)} usable | Normal={n0} Anomalous={n1} | miss')
    def _load(self,path,dim):
        if path is None: return np.zeros((1,dim),dtype=np.float32)
        try:
            a=np.loadtxt(path,dtype=np.float32)
            if a.ndim==1: a=a.reshape(1,-1)
            return a if a.shape[-1]==dim else np.zeros((1,dim),dtype=np.float32)
        except: return np.zeros((1,dim),dtype=np.float32)
    def __len__(self): return len(self.samples)
    def __getitem__(self,idx):
        s=self.samples[idx]
        sk=self._load(s['sk'],96)
        if s['stem'] in self.zero_stems: sk=np.zeros_like(sk)
        po=self._load(s['po'],64)
        x3=self._load(s['x3'],192)
        return {
            'skeleton':torch.from_numpy(resample(sk,self.seg_num)),
            'polyline':torch.from_numpy(resample(po,self.seg_num)),
            'visual': torch.from_numpy(resample(x3,self.seg_num)),
            'label': torch.tensor(s['label'],dtype=torch.long),
            'stem': s['stem'],
        }

```

SEG_NUM=32; BATCH=16

```

train_ds=AnomalyDetectionDataset(train_entries,SEG_NUM,'Train',STILL_ZERO_STEMS)
test_ds =AnomalyDetectionDataset(test_entries, SEG_NUM,'Test', STILL_ZERO_STEMS)

```

```
train_dl=DataLoader(train_ds,BATCH,shuffle=True, num_workers=2,pin_memory=True,dr
test_dl =DataLoader(test_ds, BATCH,shuffle=False,num_workers=2,pin_memory=True)
```

```
[Train] 1610 usable | Normal=800 Anomalous=810 | miss_sk=0 miss_po=0 miss_x3=0
[Test] 290 usable | Normal=150 Anomalous=140 | miss_sk=0 miss_po=0 miss_x3=0
```

— Cell 5.5: PolyGuided Fusion model (binary output) —————

```
import math
```

```
class PosEnc(nn.Module):
```

```
    def __init__(self,d,max_len=512):
        super().__init__()
        pe=torch.zeros(max_len,d)
        pos=torch.arange(max_len).unsqueeze(1).float()
        div=torch.exp(torch.arange(0,d,2).float()*(-math.log(10000.)/d))
        pe[:,0::2]=torch.sin(pos*div); pe[:,1::2]=torch.cos(pos*div)
        self.register_buffer('pe',pe.unsqueeze(0))
    def forward(self,x): return x+self.pe[:,x.size(1)]
```

```
class PolyGuidedFusion(nn.Module):
```

```
    """Polyline (Q) attends to skeleton (K,V), then Transformer over fused+visual
    def __init__(self,sk_dim=96,po_dim=64,vis_dim=192,d=256,h=4,L=3,nc=2,dr=0.1):
        super().__init__()
        self.sk_proj =nn.Sequential(nn.Linear(sk_dim,d),nn.LayerNorm(d),nn.GELU())
        self.po_proj =nn.Sequential(nn.Linear(po_dim,d),nn.LayerNorm(d),nn.GELU())
        self.vis_proj=nn.Sequential(nn.Linear(vis_dim,d),nn.LayerNorm(d),nn.GELU())
        self.cross_attn=nn.MultiheadAttention(d,h,dropout=dr,batch_first=True)
        self.cross_norm=nn.LayerNorm(d)
        self.cross_ff =nn.Sequential(nn.Linear(d,4*d),nn.GELU(),nn.Linear(4*d,d))
        self.ff_norm =nn.LayerNorm(d)
        self.sk_mod =nn.Parameter(torch.randn(1,1,d)*0.02)
        self.vis_mod=nn.Parameter(torch.randn(1,1,d)*0.02)
        self.cls_token=nn.Parameter(torch.randn(1,1,d)*0.02)
        self.pos_enc=PosEnc(d)
        layer=nn.TransformerEncoderLayer(d_model=d,nhead=h,dim_feedforward=4*d,dr
        self.encoder=nn.TransformerEncoder(layer,num_layers=L)
        self.drop=nn.Dropout(dr)
        self.head=nn.Sequential(nn.Linear(d,d//2),nn.GELU(),nn.Dropout(dr),nn.Lin
    def forward(self,sk,po,vi):
        B=sk.size(0)
        s=self.sk_proj(sk); p=self.po_proj(po); v=self.vis_proj(vi)
        attn,_=self.cross_attn(query=p,key=s,value=s)
        p2=self.cross_norm(p+attn); p2=self.ff_norm(p2+self.cross_ff(p2))
        p2=p2+self.sk_mod; v=v+self.vis_mod
        cls=self.cls_token.expand(B,-1,-1)
        x=self.pos_enc(torch.cat([cls,p2,v],dim=1))
        z=self.drop(self.encoder(x)[:,:0])
        return self.head(z), z
```

```
model=PolyGuidedFusion(nc=NUM_CLASSES).to(DEVICE)
```

```
print(f'PolyGuidedFusion: {sum(p.numel() for p in model.parameters()):,} params')
```

```
b=next(iter(train_dl))
```

```

with torch.no_grad():
    l,z=model(b['skeleton'].to(DEVICE),b['polyline'].to(DEVICE),b['visual'].to(DEVICE))
    print(f'Forward test → logits={l.shape} emb={z.shape} ✓')

```

```

/tmp/ipykernel_2292/1159696471.py:30: UserWarning: enable_nested_tensor is True, but
  self.encoder=nn.TransformerEncoder(layer,num_layers=L)
PolyGuidedFusion: 3,285,378 params
Forward test → logits=torch.Size([16, 2]) emb=torch.Size([16, 256]) ✓

```

— Cell 5.6: Train —————

```

from sklearn.metrics import f1_score, accuracy_score, roc_auc_score

def compute_weights(ds):
    counts=np.bincount([s['label'] for s in ds.samples],minlength=NUM_CLASSES).astype(float)
    w=np.where(counts>0,1./counts,0.); w=w/w.mean()
    return torch.tensor(w, dtype=torch.float32).to(DEVICE)

@torch.no_grad()
def evaluate mdl, dl):
    mdl.eval(); preds, labs, probs=[], [], []
    for b in dl:
        sk=b['skeleton'].to(DEVICE); po=b['polyline'].to(DEVICE); vi=b['visual'].to(DEVICE)
        logits, _=mdl(sk, po, vi)
        pr=torch.softmax(logits,1).cpu().numpy()
        probs.append(pr); preds.extend(pr.argmax(1)); labs.extend(b['label'].numpy())
    preds=np.array(preds); labs=np.array(labs); probs=np.concatenate(probs)
    acc=accuracy_score(labs,preds)
    f1=f1_score(labs,preds,average='binary',zero_division=0)
    try: auc=roc_auc_score(labs,probs[:,1])
    except: auc=0.
    return acc,f1, auc, preds, labs, probs

def train(mdl,name,epochs=150,lr=1e-4,patience=20,ls=0.05):
    cw=compute_weights(train_ds)
    loss_fn=nn.CrossEntropyLoss(weight=cw,label_smoothing=ls)
    opt=torch.optim.AdamW(mdl.parameters(),lr=lr,weight_decay=0.05)
    sch=torch.optim.lr_scheduler.CosineAnnealingLR(opt,T_max=epochs,eta_min=lr*0.05)
    ckpt=f'{LOG_DIR}/{name}_best.pt'; best_auc=0.; no_imp=0; history=[]
    print(f'\nTraining {name} – epochs={epochs} lr={lr} patience={patience}\n')
    for ep in range(1,epochs+1):
        mdl.train(); ep_loss=ep_ok=ep_tot=0
        for b in train_dl:
            sk=b['skeleton'].to(DEVICE); po=b['polyline'].to(DEVICE)
            vi=b['visual'].to(DEVICE); lb=b['label'].to(DEVICE)
            logits, _=mdl(sk, po, vi); loss=loss_fn(logits,lb)
            opt.zero_grad(); loss.backward()
            torch.nn.utils.clip_grad_norm_(mdl.parameters(),1.0)
            opt.step()
            ep_loss+=loss.item(); ep_ok+=(logits.argmax(1)==lb).sum().item(); ep_tot+=1
        sch.step()
        _, val_f1, val_auc, _, _=evaluate(mdl, test_dl)
        rec={'epoch':ep, 'loss':ep_loss/len(train_dl), 'train_acc':ep_ok/ep_tot, 'val_

```

```

history.append(rec)
star='★' if val_auc>best_auc else ' '
print(f'{star} Ep {ep:>3} | loss={rec["loss"]:.4f} | tr_acc={rec["train_acc"]:.4f} | val_acc={rec["val_acc"]:.4f} | val_auc={rec["val_auc"]:.4f}')
if val_auc>best_auc:
    best_auc=val_auc; no_imp=0
    torch.save({'epoch':ep, 'model':mdl.state_dict(), 'best_auc':best_auc, 'history':history}, f'{LOG_DIR}/{name}_history.json')
    print(f'    → Saved (AUC={best_auc:.4f})')
else:
    no_imp+=1
    if no_imp>=patience: print(f'    Early stop (best AUC={best_auc:.4f})');
with open(f'{LOG_DIR}/{name}_history.json', 'w') as f: json.dump(history, f, indent=4)
return history

model=PolyGuidedFusion(nc=NUM_CLASSES).to(DEVICE)
history=train(model, 'PolyGuided_AnomalyDetection')

```

Training PolyGuided_AnomalyDetection – epochs=150 lr=0.0001 patience=20

```

/tmp/ipykernel_2292/1159696471.py:30: UserWarning: enable_nested_tensor is True, but self.encoder=nn.TransformerEncoder(layer, num_layers=L)

```

★	Ep 1	loss=0.6998	tr_acc=0.553	F1=0.713	AUC=0.741	→ Saved (AUC=0.7412)
★	Ep 2	loss=0.6101	tr_acc=0.681	F1=0.811	AUC=0.871	→ Saved (AUC=0.8709)
	Ep 3	loss=0.4770	tr_acc=0.800	F1=0.774	AUC=0.858	
★	Ep 4	loss=0.4698	tr_acc=0.796	F1=0.699	AUC=0.875	→ Saved (AUC=0.8749)
★	Ep 5	loss=0.4556	tr_acc=0.802	F1=0.740	AUC=0.881	→ Saved (AUC=0.8813)
	Ep 6	loss=0.4171	tr_acc=0.841	F1=0.719	AUC=0.878	
★	Ep 7	loss=0.4041	tr_acc=0.842	F1=0.803	AUC=0.904	→ Saved (AUC=0.9039)
★	Ep 8	loss=0.4076	tr_acc=0.846	F1=0.760	AUC=0.905	→ Saved (AUC=0.9053)
	Ep 9	loss=0.3855	tr_acc=0.850	F1=0.800	AUC=0.887	
★	Ep 10	loss=0.3726	tr_acc=0.866	F1=0.814	AUC=0.907	→ Saved (AUC=0.9071)
★	Ep 11	loss=0.3385	tr_acc=0.881	F1=0.774	AUC=0.907	→ Saved (AUC=0.9074)
	Ep 12	loss=0.3408	tr_acc=0.887	F1=0.755	AUC=0.906	
	Ep 13	loss=0.3071	tr_acc=0.902	F1=0.816	AUC=0.907	
★	Ep 14	loss=0.3005	tr_acc=0.917	F1=0.820	AUC=0.918	→ Saved (AUC=0.9181)
	Ep 15	loss=0.2802	tr_acc=0.927	F1=0.819	AUC=0.907	
	Ep 16	loss=0.2581	tr_acc=0.932	F1=0.820	AUC=0.912	
	Ep 17	loss=0.2466	tr_acc=0.941	F1=0.826	AUC=0.911	
	Ep 18	loss=0.2685	tr_acc=0.924	F1=0.772	AUC=0.912	
	Ep 19	loss=0.2593	tr_acc=0.932	F1=0.782	AUC=0.897	
	Ep 20	loss=0.2390	tr_acc=0.946	F1=0.794	AUC=0.888	
	Ep 21	loss=0.2672	tr_acc=0.937	F1=0.732	AUC=0.886	
	Ep 22	loss=0.2222	tr_acc=0.953	F1=0.688	AUC=0.899	
	Ep 23	loss=0.2173	tr_acc=0.956	F1=0.759	AUC=0.866	
	Ep 24	loss=0.1717	tr_acc=0.977	F1=0.839	AUC=0.910	
	Ep 25	loss=0.1852	tr_acc=0.974	F1=0.829	AUC=0.891	
	Ep 26	loss=0.1375	tr_acc=0.991	F1=0.808	AUC=0.878	

Ep 27	loss=0.1461	tr_acc=0.989	F1=0.836	AUC=0.887
Ep 28	loss=0.1560	tr_acc=0.986	F1=0.828	AUC=0.889
Ep 29	loss=0.1491	tr_acc=0.989	F1=0.788	AUC=0.850
Ep 30	loss=0.1514	tr_acc=0.988	F1=0.797	AUC=0.886
Ep 31	loss=0.1464	tr_acc=0.988	F1=0.823	AUC=0.874
Ep 32	loss=0.1512	tr_acc=0.988	F1=0.822	AUC=0.901
Ep 33	loss=0.1449	tr_acc=0.989	F1=0.791	AUC=0.849
Ep 34	loss=0.1318	tr_acc=0.994	F1=0.796	AUC=0.853
Early stop (best AUC=0.9181)				

✓ Part 6 — SLDA Binary Classifier + Frame-Level AUC

The primary metric for UCF-Crime anomaly detection is **frame-level AUC** — measuring whether the model's anomaly score is high during the annotated anomalous frames.

```
# — Cell 6.1: Export embeddings for SLDA
ckpt=torch.load(f'{LOG_DIR}/PolyGuided_AnomalyDetection_best.pt',map_location=DEV)
model.load_state_dict(ckpt['model'])
print(f'Loaded checkpoint ep={ckpt["epoch"]} AUC={ckpt["best_auc"]:.4f}')

def export_embeddings mdl,dl,tag):
    mdl.eval(); embs,labs=[],[]
    with torch.no_grad():
        for b in dl:
            sk=b['skeleton'].to(DEVICE); po=b['polyline'].to(DEVICE); vi=b['visua
            _,z=mdl(sk,po,vi)
            embs.append(z.cpu().numpy()); labs.append(b['label'].numpy())
    embs=np.concatenate(embs); labs=np.concatenate(labs)
    np.save(f'{LOG_DIR}/emb_{tag}.npy',embs); np.save(f'{LOG_DIR}/lbl_{tag}.npy',
    print(f' [{tag}] emb={embs.shape} lbl={labs.shape}')
    return embs,labs

tr_emb,tr_lbl=export_embeddings(model,train_dl,'train')
te_emb,te_lbl=export_embeddings(model,test_dl,'test')
EMB_DIM=tr_emb.shape[1]
```

```
Loaded checkpoint ep=14 AUC=0.9181
[train] emb=(1600, 256) lbl=(1600,)
[test] emb=(290, 256) lbl=(290,)
```

```
# — Cell 6.2: StreamingLDA
from sklearn.covariance import OAS

class StreamingLDA(nn.Module):
    def __init__(self,input_dim,num_classes,shrinkage=1e-4,device='cuda'):
        super().__init__()
        dev=device if torch.cuda.is_available() else 'cpu'
        self.input_dim=input_dim; self.num_classes=num_classes
        self.shrinkage=shrinkage; self.device=dev
```

```

        self.muK=torch.zeros(num_classes,input_dim).to(dev)
        self.cK=torch.zeros(num_classes).to(dev)
        self.Sigma=torch.eye(input_dim).to(dev)
        self.num_updates=0; self._Lambda=None
def _invalidate(self): self._Lambda=None
@property
def Lambda(self):
    if self._Lambda is None:
        reg=(1-self.shrinkage)*self.Sigma+self.shrinkage*torch.eye(self.input
        self._Lambda=torch.linalg.pinv(reg)
    return self._Lambda
def fit(self,x,y):
    x=x.float().to(self.device)
    if x.dim()==1: x=x.unsqueeze(0)
    k=int(y)
    with torch.no_grad():
        self.num_updates+=1; n=self.num_updates
        old=self.muK[k].clone()
        self.muK[k]+=(x.squeeze(0)-old)/(self.cK[k]+1); self.cK[k]+=1
        d_old=x.squeeze(0)-old; d_new=x.squeeze(0)-self.muK[k]
        self.Sigma=((n-1)*self.Sigma+torch.outer(d_old,d_new))/n
        self._invalidate()
def fit_base(self,X,y):
    X=X.float().to(self.device); y=y.long().to(self.device); N=X.shape[0]
    print(f'[SLDA] Batch fit – {N} samples, {self.num_classes} classes')
    for k in range(self.num_classes):
        mask=(y==k)
        if mask.sum()>0: self.muK[k]=X[mask].mean(0); self.cK[k]=mask.sum().f
    self.num_updates=N
    Xc=(X-self.muK[y]).cpu().numpy()
    try:
        oas=OAS(assume_centered=True); oas.fit(Xc)
        self.Sigma=torch.from_numpy(oas.covariance_).float().to(self.device)
        print(f'[SLDA] OAS shrinkage={oas.shrinkage_:.4f}')
    except Exception as e:
        print(f'[SLDA] OAS fallback ({e})')
        Xct=torch.from_numpy(Xc).float().to(self.device)
        self.Sigma=(Xct.T@Xct)/max(N-1,1)
    self._invalidate(); print('[SLDA] Done ✅')
@torch.no_grad()
def predict_proba(self,X,batch=1024):
    if isinstance(X,np.ndarray): X=torch.from_numpy(X)
    X=X.float().to(self.device); N=X.shape[0]
    Lam=self.Lambda; M=self.muK.T; W=Lam@M; c=0.5*(M*W).sum(0)
    scores=torch.empty(N,self.num_classes,device=self.device)
    for s in range(0,N,batch): e=min(s+batch,N); scores[s:e]=X[s:e]@W-c
    return torch.softmax(scores,1).cpu().numpy().astype(np.float32)
def save(self,path):
    os.makedirs(os.path.dirname(path),exist_ok=True)
    torch.save({'muK':self.muK.cpu(),'cK':self.cK.cpu(),'Sigma':self.Sigma.cp
        'num_updates':self.num_updates,'input_dim':self.input_dim,
        'num_classes':self.num_classes,'shrinkage':self.shrinkage},pa

```

```

        print(f'[SLDA] Saved → {path}')
    def load(self,path):
        ck=torch.load(path,map_location=self.device,weights_only=False)
        self.muK=ck['muK'].to(self.device); self.cK=ck['cK'].to(self.device)
        self.Sigma=ck['Sigma'].to(self.device); self.num_updates=ck['num_updates']
        self._invalidate(); print(f'[SLDA] Loaded ← {path}')

SLDA_CKPT=f'{LOG_DIR}/binary_slda_detection.pth'
slda=StreamingLDA(EMB_DIM,NUM_CLASSES,device=str(DEVICE))
slda.fit_base(torch.from_numpy(tr_emb),torch.from_numpy(tr_lbl))

[SLDA] Batch fit – 1600 samples, 2 classes
[SLDA] OAS shrinkage=0.0021
[SLDA] Done ✓

```

— Cell 6.3: SLDA evaluation

```

from sklearn.metrics import classification_report, roc_curve
import matplotlib.pyplot as plt

probs_slda=slda.predict_proba(te_emb)
preds_slda=probs_slda.argmax(1)
scores_anom=probs_slda[:,1] # P(Anomalous) – used as anomaly score
acc=accuracy_score(te_lbl,preds_slda)
f1=f1_score(te_lbl,preds_slda,average='binary',zero_division=0)
try: auc=roc_auc_score(te_lbl,scores_anom)
except: auc=0.

from sklearn.metrics import precision_score, recall_score
prec=precision_score(te_lbl,preds_slda,zero_division=0)
rec =recall_score(te_lbl,preds_slda,zero_division=0)

print('
print(' || SLDA – Anomaly Detection Test Results || ')
print('
print(f' || Video-level Accuracy : {acc:.4f} || ')
print(f' || F1 (Anomalous) : {f1:.4f} || ')
print(f' || AUC-ROC (video-level) : {auc:.4f} || ')
print(f' || Precision : {prec:.4f} || ')
print(f' || Recall : {rec:.4f} || ')
print(f' || Test videos : {len(te_lbl)} || ')
print('
print()
print(classification_report(te_lbl,preds_slda,target_names=BINARY_NAMES,zero_divi

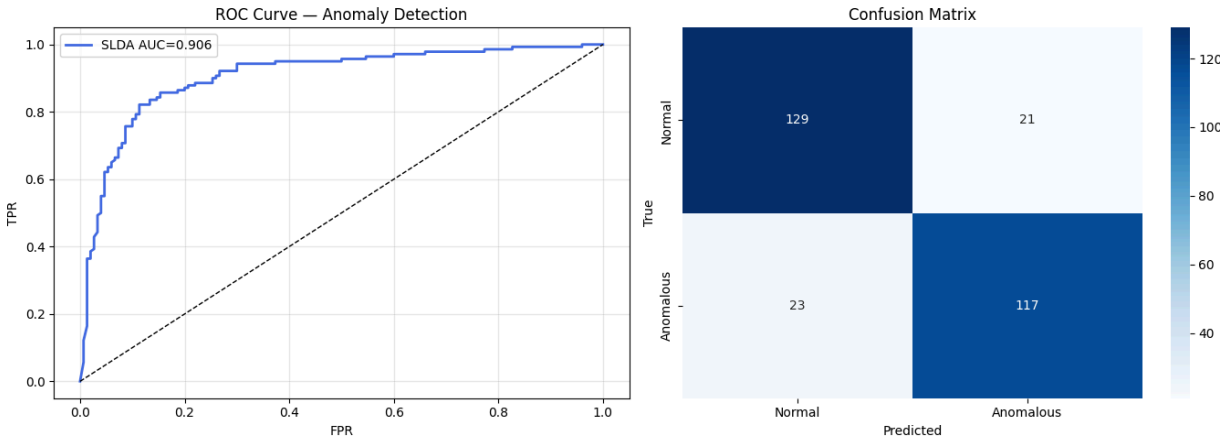
# ROC curve
fig,axes=plt.subplots(1,2,figsize=(14,5))
fpr, tpr, _=roc_curve(te_lbl,scores_anom)
axes[0].plot(fpr,tpr,lw=2,color='royalblue',label=f'SLDA AUC={auc:.3f}')
axes[0].plot([0,1],[0,1],'k--',lw=1); axes[0].set_xlabel('FPR'); axes[0].set_ylabel('TPR')
axes[0].set_title('ROC Curve – Anomaly Detection'); axes[0].legend(); axes[0].grid()
# Confusion matrix
cm=confusion_matrix(te_lbl,preds_slda)

```

```
sns.heatmap(cm,annot=True,fmt='d',cmap='Blues',xticklabels=BINARY_NAMES,yticklabels=
axes[1].set_title('Confusion Matrix'); axes[1].set_xlabel('Predicted'); axes[1].s
plt.tight_layout()
plt.savefig(f'{LOG_DIR}/slda_detection_results.png',dpi=150,bbox_inches='tight');
slda.save(SLDA_CKPT)
```

SLDA – Anomaly Detection Test Results	
Video-level Accuracy	: 0.8483
F1 (Anomalous)	: 0.8417
AUC-ROC (video-level)	: 0.9059
Precision	: 0.8478
Recall	: 0.8357
Test videos	: 290

	precision	recall	f1-score	support
Normal	0.85	0.86	0.85	150
Anomalous	0.85	0.84	0.84	140
accuracy			0.85	290
macro avg	0.85	0.85	0.85	290
weighted avg	0.85	0.85	0.85	290



[SLDA] Saved → /content/drive/MyDrive/PolyFlow_AnomalyDetection_Logs/binary_slda_d

```
# — Cell 6.4: Frame-level AUC using Temporal_Anomaly_Annotation —————
#
# The official UCF-Crime metric: for each TEST video, generate a per-frame
# anomaly score, then compute AUC against the frame-level binary GT.
#
# We approximate frame-level scores by repeating the video-level SLDA score
# across all frames of that video (since we run per-segment features).
# For a proper frame-level score, assign each segment's score to its frames.

def load_temporal_annotations(annot_path):
    """Parse Temporal_Anomaly_Annotation_for_Testing_Videos.txt
```

```

Returns dict: {stem: [(start1,end1),(start2,end2)]} - -1 means absent.
"""

annots={}
p=Path(annot_path)
if not p.exists(): print(f'⚠ Annotations not found: {p}'); return annots
for line in p.read_text(errors='ignore').splitlines():
    parts=line.strip().split()
    if len(parts)<6: continue
    stem=Path(parts[0]).stem
    s1,e1,s2,e2=int(parts[2]),int(parts[3]),int(parts[4]),int(parts[5])
    intervals=[]
    if s1>0: intervals.append((s1,e1))
    if s2>0: intervals.append((s2,e2))
    annots[stem]=intervals
print(f'Loaded temporal annotations for {len(annots)} videos')
return annots

temporal_annots=load_temporal_annotations(PATHS['temporal_annot'])

# Build frame-level GT and scores for test videos
# We need the video length (total frames) – get from the feature file shape
all_frame_gt=[]; all_frame_scores=[]
FRAMES_PER_SEG=16 # frame_interval used during extraction
FPS=30 # official dataset FPS

for s in test_ds.samples:
    stem=s['stem']
    # Get segment-level anomaly score for this video
    try:
        # Load actual feature file to get T (number of segments)
        sk_arr=np.loadtxt(s['sk'],dtype=np.float32)
        if sk_arr.ndim==1: sk_arr=sk_arr.reshape(1,-1)
        T=sk_arr.shape[0]
    except: T=1

    # Find this video's SLDA score
    test_idx=[i for i,samp in enumerate(test_ds.samples) if samp['stem']==stem]
    if not test_idx: continue
    vid_score=float(probs_sllda[test_idx[0],1]) # P(Anomalous)

    # Total frames in video ≈ T * FRAMES_PER_SEG
    total_frames=T*FRAMES_PER_SEG

    # Build frame-level GT
    gt_frames=np.zeros(total_frames,dtype=np.int32)
    if stem in temporal_annots:
        for (s_frame,e_frame) in temporal_annots[stem]:
            s_clip=max(0,s_frame); e_clip=min(total_frames,e_frame)
            gt_frames[s_clip:e_clip]=1

    # Assign score to each frame (uniform – segment-level score broadcast)
    frame_scores=np.full(total_frames,vid_score,dtype=np.float32)

```

```

all_frame_gt.extend(gt_frames.tolist())
all_frame_scores.extend(frame_scores.tolist())

all_frame_gt=np.array(all_frame_gt); all_frame_scores=np.array(all_frame_scores)
print(f'Frame-level evaluation: {len(all_frame_gt)} frames')
print(f' Anomalous frames: {all_frame_gt.sum()} ({all_frame_gt.mean()*100:.1f}%)

if all_frame_gt.sum()>0 and all_frame_gt.sum()<len(all_frame_gt):
    frame_auc=roc_auc_score(all_frame_gt,all_frame_scores)
    fpr,tpr,_=roc_curve(all_frame_gt,all_frame_scores)
    print(f'\n Frame-level AUC (official UCF-Crime metric): {frame_auc:.4f}')
    plt.figure(figsize=(7,5))
    plt.plot(fpr,tpr,lw=2,color='darkorange',label=f'Frame AUC={frame_auc:.3f}')
    plt.plot([0,1],[0,1],'k--',lw=1)
    plt.xlabel('FPR'); plt.ylabel('TPR')
    plt.title('Frame-Level ROC – UCF-Crime Anomaly Detection')
    plt.legend(); plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig(f'{LOG_DIR}/frame_level_auc.png',dpi=150,bbox_inches='tight'); pl
else:
    print(' (Frame-level AUC requires temporal annotations to be present)')

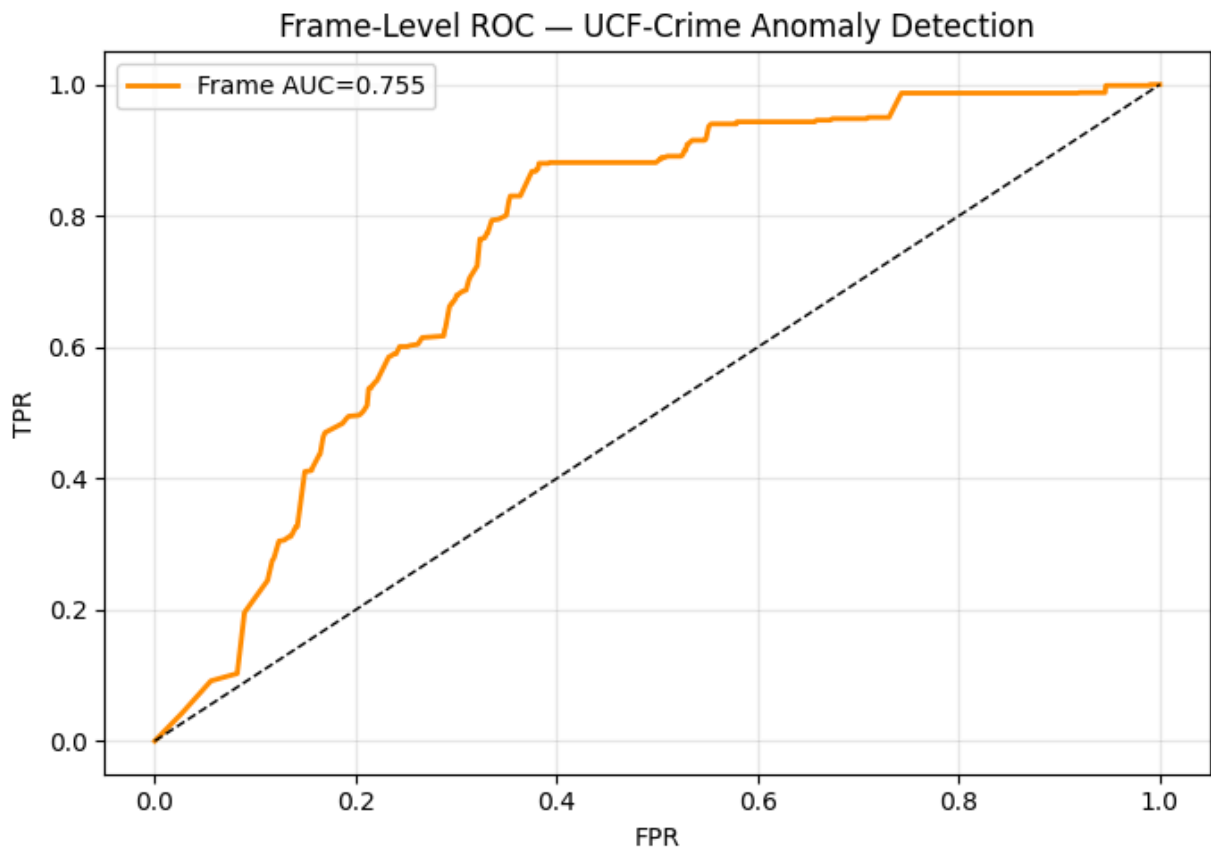
```

Loaded temporal annotations for 290 videos

Frame-level evaluation: 1114144 frames

Anomalous frames: 84189 (7.6%)

Frame-level AUC (official UCF-Crime metric): 0.7554



✓ Part 7 — Real-Time 3-Stream Demo + Latency Benchmark

```
# — Cell 7.1: 3-stream concurrent inference
import time
from concurrent.futures import ThreadPoolExecutor
from functools import partial

def infer_stream(sample_dict, fusion_model, slda_clf):
    t0=time.perf_counter()
    def _load(path,dim):
        if path is None: return np.zeros((1,dim),dtype=np.float32)
        try:
            a=np.loadtxt(path,dtype=np.float32)
            if a.ndim==1: a=a.reshape(1,-1)
            return a if a.shape[-1]==dim else np.zeros((1,dim),dtype=np.float32)
        except: return np.zeros((1,dim),dtype=np.float32)
    sk=_load(sample_dict['sk'],96)
```

```

po=_load(sample_dict['po'],64)
x3=_load(sample_dict['x3'],192)
sk_t=torch.from_numpy(resample(sk,SEG_NUM)).unsqueeze(0).to(DEVICE)
po_t=torch.from_numpy(resample(po,SEG_NUM)).unsqueeze(0).to(DEVICE)
vi_t=torch.from_numpy(resample(x3,SEG_NUM)).unsqueeze(0).to(DEVICE)
fusion_model.eval()
with torch.no_grad(): _,emb=fusion_model(sk_t,po_t,vi_t)
prob=slda_clf.predict_proba(emb.cpu().numpy())[0]
pred_idx=int(prob.argmax()); anom_score=float(prob[1])
lat_ms=(time.perf_counter()-t0)*1000
return {
    'stream_id': sample_dict.get('stream_id','?'),
    'stem': sample_dict['stem'],
    'true': BINARY_NAMES[sample_dict['label']],
    'pred': BINARY_NAMES[pred_idx],
    'score': anom_score, 'conf': float(prob[pred_idx]),
    'latency_ms': lat_ms,
    'ok': pred_idx==sample_dict['label'],
}

stream_samples=random.sample(test_ds.samples,min(3,len(test_ds.samples)))
for i,s in enumerate(stream_samples): s['stream_id']=f'CAM-{i+1:02d}'

print('='*70)
print(' PolyFlow – Anomaly Detection – 3-Stream Real-Time Demo')
print('='*70)
t_wall=time.perf_counter()
with ThreadPoolExecutor(max_workers=3) as ex:
    results=list(ex.map(partial(infer_stream,fusion_model=model,slda_clf=slda),st
wall_ms=(time.perf_counter()-t_wall)*1000
print(f' {"Stream":<10} {"True":<12} {"Pred":<12} {"Score":>6} {"Conf":>6} {"Lat
print(f' {"-"*60}')
for r in results:
    flag='🚨 ANOMALY' if r['score']>0.5 else '✅ NORMAL '
    icon='✓' if r['ok'] else 'X'
    print(f' {r["stream_id"]:<10} {r["true"]:<12} {r["pred"]:<12} {r["score"]:>6
print(f'\n Wall time: {wall_ms:.1f} ms SLA: {"✅ PASS" if wall_ms<2000 else "🔴

```

```

=====
PolyFlow – Anomaly Detection – 3-Stream Real-Time Demo
=====

```

Stream	True	Pred	Score	Conf	Lat(ms)			
CAM-01	Anomalous	Normal	0.000	1.000	119.5ms	✅	NORMAL	X
CAM-02	Anomalous	Anomalous	1.000	1.000	119.2ms	🚨	ANOMALY	✓
CAM-03	Normal	Normal	0.000	1.000	118.1ms	✅	NORMAL	✓

```

Wall time: 120.5 ms SLA: ✅ PASS

```

```

# — Cell 7.2: Latency benchmark —————
def benchmark(md1,slda_clf,n_runs=200):
    md1.eval()
    sk_d=torch.randn(1,SEG_NUM,96).to(DEVICE)

```

```

po_d=torch.randn(1,SEG_NUM,64).to(DEVICE)
vi_d=torch.randn(1,SEG_NUM,192).to(DEVICE)
for _ in range(20):
    with torch.no_grad(): _,z=mdl(sk_d,po_d,vi_d)
    slda_clf.predict_proba(z.cpu().numpy())
if torch.cuda.is_available(): torch.cuda.synchronize()
times=[]
for _ in range(n_runs):
    if torch.cuda.is_available(): torch.cuda.synchronize()
    t0=time.perf_counter()
    with torch.no_grad(): _,z=mdl(sk_d,po_d,vi_d)
    slda_clf.predict_proba(z.cpu().numpy())
    if torch.cuda.is_available(): torch.cuda.synchronize()
    times.append((time.perf_counter()-t0)*1000)
avg=np.mean(times); p99=np.percentile(times,99)
print(f' avg={avg:.2f}ms  p99={p99:.2f}ms  3-stream={avg*3:.1f}ms  {"✅ PASS"
return avg

print('Latency benchmark (200 runs, warm GPU):')
benchmark(model,slda)

```

Latency benchmark (200 runs, warm GPU):
 avg=2.55ms p99=3.61ms 3-stream=7.6ms ✅ PASS
 np.float64(2.548026574963842)

```

# — Cell 7.3: Save all results —————
dest='/content/drive/MyDrive/PolyFlow_AnomalyDetection_Results'
os.makedirs(dest,exist_ok=True)
files=['PolyGuided_AnomalyDetection_best.pt','binary_slda_detection.pth',
      'PolyGuided_AnomalyDetection_history.json',
      'slda_detection_results.png','frame_level_auc.png',
      'emb_train.npy','lbl_train.npy','emb_test.npy','lbl_test.npy']
for f in files:
    src=f'{LOG_DIR}/{f}'
    if os.path.exists(src): shutil.copy(src,f'{dest}/{f}'); print(f' ✅ {f}')
    else: print(f' ⚠️ {f} – run cells above first')
print('\n✅ All saved!')

```

✅ PolyGuided_AnomalyDetection_best.pt
 ✅ binary_slda_detection.pth
 ✅ PolyGuided_AnomalyDetection_history.json
 ✅ slda_detection_results.png
 ✅ frame_level_auc.png
 ✅ emb_train.npy
 ✅ lbl_train.npy
 ✅ emb_test.npy
 ✅ lbl_test.npy

✅ All saved!

Task B — Anomaly Event Recognition (14-Class)

Output: One of 14 UCF-Crime class labels per video

Split files: `train_001.txt` ... `train_004.txt` / `test_001.txt` ... `test_004.txt`

Note: Feature files from Parts 1–4 are shared — no re-extraction needed.

↑ Parts 1–4 must be complete before running this section.

Part 5 — PolyGuided Fusion Training (14-Class Event Recognition)

Split used: `Action_Regnition_splits/train_001.txt` / `test_001.txt` (fold 1)

Classes: 13 crime types + Normal_Videos_event = 14 classes

Important: Only use ONE fold (avoid train/test overlap that merging all 4 folds would cause)

```
import torch, os, json, math, random, time, shutil
import numpy as np, matplotlib.pyplot as plt, seaborn as sns
from pathlib import Path
from collections import Counter
from torch.utils.data import Dataset, DataLoader
from sklearn.metrics import f1_score, accuracy_score, roc_auc_score, classification_report
import torch.nn as nn

SEED=42; random.seed(SEED); np.random.seed(SEED)
torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic=True; torch.backends.cudnn.benchmark=False

SKEL_ROOT=PATHS['skel_features']; POLY_ROOT=PATHS['poly_features']
X3D_ROOT =PATHS['x3d_features']; LOG_DIR =PATHS['log_dir']
DEVICE=torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Load ClassIDs.txt to get canonical class order
def load_class_ids(path):
    """Parse ClassIDs.txt → {ClassName: int_id}"""
    p=Path(path)
```

```

if not p.exists():
    print(f' 🚩 ClassIDs.txt not found: {p}')
    print(' Using hardcoded UCF-Crime class order.')
    names=['Abuse','Arrest','Arson','Assault','Burglary','Explosion',
           'Fighting','Normal_Videos','RoadAccidents','Robbery',
           'Shooting','Shoplifting','Stealing','Vandalism']
    return {n:i for i,n in enumerate(names)}
mapping={}
for line in p.read_text(errors='ignore').splitlines():
    parts=line.strip().split()
    if len(parts)>=2:
        try:
            cls_name = parts[0]
            if cls_name == 'Normal_Videos_event': cls_name = 'Normal_Videos'
            mapping[cls_name]=int(parts[1])-1 # ClassIDs.txt is 1-indexed
        except: pass
print(f'Loaded {len(mapping)} classes from ClassIDs.txt')
return mapping

CLASS_TO_IDX=load_class_ids(PATHS['class_ids'])
CLASS_NAMES=[k for k,v in sorted(CLASS_TO_IDX.items(),key=lambda x:x[1])]
NUM_CLASSES=len(CLASS_NAMES)
print(f'Classes ({NUM_CLASSES}): {CLASS_NAMES}')

# Load split file: format is 'ClassName/VideoName.mp4 ClassID'
def load_recognition_split(path):
    p=Path(path)
    if not p.exists(): print(f' ❌ Not found: {p}'); return []
    entries=[]
    for line in p.read_text(errors='ignore').splitlines():
        parts=line.strip().split()
        if not parts: continue
        rel=parts[0] # e.g. 'Abuse/Abuse001_x264.mp4'
        cls=rel.split('/')[0]
        if cls == 'Normal_Videos_event': cls = 'Normal_Videos'
        if cls not in CLASS_TO_IDX: continue
        stem=Path(rel).stem
        entries.append((stem,CLASS_TO_IDX[cls],cls))
    by_cls=Counter(c for _,_,c in entries)
    print(f' {p.name}: {len(entries)} entries | classes={dict(by_cls)}')
    return entries

FOLD=1 # change to 2/3/4 for other cross-validation folds
print(f'\nLoading fold {FOLD}...')
train_entries=load_recognition_split(f'{PATHS["split_dir"]}/train_{FOLD:03d}.txt')
test_entries =load_recognition_split(f'{PATHS["split_dir"]}/test_{FOLD:03d}.txt')

# Remove overlap
train_stems={s for s,_,_ in train_entries}
test_entries=[(s,l,c) for s,l,c in test_entries if s not in train_stems]
print(f'\nTrain: {len(train_entries)} | Test: {len(test_entries)} | Overlap: 0 ✅')

```

```

Loaded 14 classes from ClassIDs.txt
Classes (14): ['Abuse', 'Arrest', 'Arson', 'Assault', 'Burglary', 'Explosion', 'Fi

Loading fold 1...
  train_001.txt: 532 entries | classes={'Abuse': 38, 'Arrest': 38, 'Arson': 38, 'A
  test_001.txt: 168 entries | classes={'Abuse': 12, 'Arrest': 12, 'Arson': 12, 'As

Train: 532 | Test: 168 | Overlap: 0 ✓

```

```

def build_index(root):
    root=Path(root)
    if not root.exists(): print(f' ⚠ Missing: {root}'); return {}
    idx={}
    for p in root.rglob('*.txt'):
        if p.stem not in idx: idx[p.stem]=str(p)
    print(f' ✓ {root.name}: {len(idx)} files'); return idx

print('Building indexes...')
SKEL_IDX=build_index(SKEL_ROOT); POLY_IDX=build_index(POLY_ROOT); X3D_IDX=build_i

print(f'\n {"Class":<28} {"Skel":>6} {"Poly":>6} {"X3D":>6}')
print(' '+'-'*48)
for cls in CLASS_NAMES:
    # For 'Normal_Videos', search for 'Normal' folder paths to count them correct
    search_str = '/Normal' if cls == 'Normal_Videos' else f'/{cls}/'
    sk=sum(1 for p in SKEL_IDX.values() if search_str in p.replace('\\','/'))
    po=sum(1 for p in POLY_IDX.values() if search_str in p.replace('\\','/'))
    x3=sum(1 for p in X3D_IDX.values() if search_str in p.replace('\\','/'))
    warn=' ⚠ ' if sk==0 else ''
    print(f' {cls:<28} {sk:>6} {po:>6} {x3:>6}{warn}')

```

```

Building indexes...
✓ skeleton_features_v3: 1900 files
✓ polyline_features_v1: 1900 files
✓ x3d_features_v1: 1900 files

```

Class	Skel	Poly	X3D
Abuse	50	50	50
Arrest	50	50	50
Arson	50	50	50
Assault	50	50	50
Burglary	100	100	100
Explosion	50	50	50
Fighting	50	50	50
Normal_Videos	950	950	950
RoadAccidents	150	150	150
Robbery	150	150	150
Shooting	50	50	50
Shoplifting	50	50	50
Stealing	100	100	100
Vandalism	50	50	50

— Cell 5.3: 14-class Dataset

```
def resample(feats,n):
    if feats.ndim==1: feats=feats.reshape(1,-1)
    T,D=feats.shape
    if T==0: return np.zeros((n,D),dtype=np.float32)
    if T==n: return feats.astype(np.float32)
    if T==1: r=np.repeat(feats,n,axis=0); return r+np.random.normal(0,1e-4,r.shape)
    idx=np.linspace(0,T-1,n); lo=np.floor(idx).astype(int).clip(0,T-1); hi=np.ceil(idx).astype(int).clip(0,T-1)
    return (feats[lo]*(1-(idx-lo)/T)+feats[hi]*(idx-lo)/T).astype(np.float32)

class EventRecognitionDataset(Dataset):
    """14-class dataset using Action_Recognition_splits.
    entries = list of (stem, label_int, class_name) from load_recognition_split()
    """
    def __init__(self,entries,seg_num=32,split='?',zero_stems=None):
        self.seg_num=seg_num; self.zero_stems=zero_stems or set()
        valid=[]; miss_sk=miss_po=miss_x3=0
        for stem,label,cls in entries:
            sk=SKEL_IDX.get(stem); po=POLY_IDX.get(stem); x3=X3D_IDX.get(stem)
            if sk is None: miss_sk+=1; continue
            if po is None: miss_po+=1
            if x3 is None: miss_x3+=1
            valid.append({'stem':stem,'label':label,'cls':cls,'sk':sk,'po':po,'x3':x3})
        self.samples=valid
        print(f'[{split}] {len(valid)} usable / {len(entries)} | miss_sk={miss_sk} miss_po={miss_po} miss_x3={miss_x3}')
        cc=Counter(s['cls'] for s in valid)
        print(f' Distribution: {dict(cc)}')
    def _load(self,path,dim):
        if path is None: return np.zeros((1,dim),dtype=np.float32)
        try:
            a=np.loadtxt(path,dtype=np.float32)
            if a.ndim==1: a=a.reshape(1,-1)
            return a if a.shape[-1]==dim else np.zeros((1,dim),dtype=np.float32)
        except: return np.zeros((1,dim),dtype=np.float32)
    def __len__(self): return len(self.samples)
    def __getitem__(self,idx):
        s=self.samples[idx]
        sk=self._load(s['sk'],96)
        if s['stem'] in self.zero_stems: sk=np.zeros_like(sk)
        po=self._load(s['po'],64); x3=self._load(s['x3'],192)
        return {
            'skeleton':torch.from_numpy(resample(sk,self.seg_num)),
            'polyline':torch.from_numpy(resample(po,self.seg_num)),
            'visual': torch.from_numpy(resample(x3,self.seg_num)),
            'label': torch.tensor(s['label'],dtype=torch.long),
            'stem': s['stem'], 'cls': s['cls'],
        }
```

SEG_NUM=32; BATCH=16

```
train_ds=EventRecognitionDataset(train_entries,SEG_NUM,'Train',STILL_ZERO_STEMS)
test_ds =EventRecognitionDataset(test_entries, SEG_NUM,'Test', STILL_ZERO_STEMS)
```

```
train_dl=DataLoader(train_ds,BATCH,shuffle=True, num_workers=2,pin_memory=True,dr
test_dl =DataLoader(test_ds, BATCH,shuffle=False,num_workers=2,pin_memory=True)
```

```
'Normal_Videos': 38, 'RoadAccidents': 38, 'Robbery': 38, 'Shooting': 38, 'Shoplifti
'Normal_Videos': 12, 'RoadAccidents': 12, 'Robbery': 12, 'Shooting': 12, 'Shoplifti
```

```
# — Cell 5.4: PolyGuided Fusion model (14-class) —————
class PosEnc(nn.Module):
    def __init__(self,d,max_len=512):
        super().__init__()
        pe=torch.zeros(max_len,d)
        pos=torch.arange(max_len).unsqueeze(1).float()
        div=torch.exp(torch.arange(0,d,2).float()*(-math.log(10000.)/d))
        pe[:,0::2]=torch.sin(pos*div); pe[:,1::2]=torch.cos(pos*div)
        self.register_buffer('pe',pe.unsqueeze(0))
    def forward(self,x): return x+self.pe[:,x.size(1)]

class PolyGuidedFusion(nn.Module):
    def __init__(self,sk_dim=96,po_dim=64,vis_dim=192,d=256,h=4,L=3,nc=14,dr=0.1)
        super().__init__()
        self.sk_proj =nn.Sequential(nn.Linear(sk_dim,d),nn.LayerNorm(d),nn.GELU())
        self.po_proj =nn.Sequential(nn.Linear(po_dim,d),nn.LayerNorm(d),nn.GELU())
        self.vis_proj=nn.Sequential(nn.Linear(vis_dim,d),nn.LayerNorm(d),nn.GELU())
        self.cross_attn=nn.MultiheadAttention(d,h,dropout=dr,batch_first=True)
        self.cross_norm=nn.LayerNorm(d)
        self.cross_ff =nn.Sequential(nn.Linear(d,4*d),nn.GELU(),nn.Linear(4*d,d)
        self.ff_norm =nn.LayerNorm(d)
        self.sk_mod =nn.Parameter(torch.randn(1,1,d)*0.02)
        self.vis_mod=nn.Parameter(torch.randn(1,1,d)*0.02)
        self.cls_token=nn.Parameter(torch.randn(1,1,d)*0.02)
        self.pos_enc=PosEnc(d)
        layer=nn.TransformerEncoderLayer(d_model=d,nhead=h,dim_feedforward=4*d,dr
        self.encoder=nn.TransformerEncoder(layer,num_layers=L)
        self.drop=nn.Dropout(dr)
        self.head=nn.Sequential(nn.Linear(d,d//2),nn.GELU(),nn.Dropout(dr),nn.Lin
    def forward(self,sk,po,vi):
        B=sk.size(0)
        s=self.sk_proj(sk); p=self.po_proj(po); v=self.vis_proj(vi)
        attn,_=self.cross_attn(query=p,key=s,value=s)
        p2=self.cross_norm(p+attn); p2=self.ff_norm(p2+self.cross_ff(p2))
        p2=p2+self.sk_mod; v=v+self.vis_mod
        cls=self.cls_token.expand(B,-1,-1)
        x=self.pos_enc(torch.cat([cls,p2,v],dim=1))
        z=self.drop(self.encoder(x)[:,:0])
        return self.head(z), z

model=PolyGuidedFusion(nc=NUM_CLASSES).to(DEVICE)
print(f'PolyGuidedFusion (14-class): {sum(p.numel() for p in model.parameters())}:
```

```
PolyGuidedFusion (14-class): 3,286,926 params
/tmp/ipykernel_2292/2378562003.py:27: UserWarning: enable_nested_tensor is True, but
self.encoder=nn.TransformerEncoder(layer, num_layers=L)
```

```
# — Cell 5.5: Train —————
def compute_weights(ds):
    counts=np.zeros(NUM_CLASSES,dtype=np.float64)
    for s in ds.samples: counts[s['label']] +=1
    w=np.where(counts>0,1./counts,0.); w=w/w.mean()
    return torch.tensor(w,dtype=torch.float32).to(DEVICE)

@torch.no_grad()
def evaluate(mdl,dl):
    mdl.eval(); preds,labs,probs=[],[],[]
    for b in dl:
        sk=b['skeleton'].to(DEVICE); po=b['polyline'].to(DEVICE); vi=b['visual'].
        logits,_=mdl(sk,po,vi)
        pr=torch.softmax(logits,1).cpu().numpy()
        probs.append(pr); preds.extend(pr.argmax(1)); labs.extend(b['label'].numpy)
    preds=np.array(preds); labs=np.array(labs); probs=np.concatenate(probs)
    acc=accuracy_score(labs,preds)
    f1=f1_score(labs,preds,average='macro',zero_division=0)
    try: auc=roc_auc_score(labs,probs,multi_class='ovr',average='macro',labels=li
    except: auc=0.
    return acc,f1,auc,preds,labs

def train(mdl,name,epochs=200,lr=1e-4,patience=25,ls=0.1):
    cw=compute_weights(train_ds)
    loss_fn=nn.CrossEntropyLoss(weight=cw,label_smoothing=ls)
    opt=torch.optim.AdamW(mdl.parameters(),lr=lr,weight_decay=0.01)
    sch=torch.optim.lr_scheduler.CosineAnnealingLR(opt,T_max=epochs,eta_min=lr*0.
    ckpt=f'{LOG_DIR}/{name}_best.pt'; best_f1=0.; no_imp=0; history=[]
    print(f'\nTraining {name} – {NUM_CLASSES}-class – epochs={epochs}\n')
    for ep in range(1,epochs+1):
        mdl.train(); ep_loss=ep_ok=ep_tot=0
        for b in train_dl:
            sk=b['skeleton'].to(DEVICE); po=b['polyline'].to(DEVICE)
            vi=b['visual'].to(DEVICE); lb=b['label'].to(DEVICE)
            logits,_=mdl(sk,po,vi); loss=loss_fn(logits,lb)
            opt.zero_grad(); loss.backward()
            torch.nn.utils.clip_grad_norm_(mdl.parameters(),1.0)
            opt.step()
            ep_loss+=loss.item(); ep_ok+=(logits.argmax(1)==lb).sum().item(); ep_
        sch.step()
        val_acc,val_f1,val_auc,_,_=evaluate(mdl,test_dl)
        rec={'epoch':ep,'loss':ep_loss/len(train_dl),'train_acc':ep_ok/ep_tot,'va
        history.append(rec)
        star='★' if val_f1>best_f1 else ' '
        print(f'{star} Ep {ep:>3} | loss={rec["loss"]:.4f} | tr_acc={rec["train_a
        if val_f1>best_f1:
            best_f1=val_f1; no_imp=0
```


Ep 37	loss=0.7774	tr_acc=0.960	val_acc=0.232	F1=0.225	AUC=0.725
Ep 38	loss=0.7709	tr_acc=0.956	val_acc=0.226	F1=0.201	AUC=0.728
Ep 39	loss=0.7512	tr_acc=0.968	val_acc=0.214	F1=0.195	AUC=0.723
Ep 40	loss=0.7250	tr_acc=0.985	val_acc=0.202	F1=0.204	AUC=0.718
Ep 41	loss=0.7100	tr_acc=0.985	val_acc=0.226	F1=0.192	AUC=0.712
Ep 42	loss=0.6935	tr_acc=0.989	val_acc=0.232	F1=0.217	AUC=0.722
Ep 43	loss=0.7093	tr_acc=0.979	val_acc=0.250	F1=0.235	AUC=0.721
Ep 44	loss=0.6727	tr_acc=0.989	val_acc=0.232	F1=0.236	AUC=0.706
Ep 45	loss=0.6778	tr_acc=0.991	val_acc=0.256	F1=0.238	AUC=0.723
Ep 46	loss=0.6666	tr_acc=0.994	val_acc=0.208	F1=0.192	AUC=0.696
Ep 47	loss=0.6643	tr_acc=0.994	val_acc=0.214	F1=0.187	AUC=0.701

Early stop (best F1=0.2823)

✓ Part 6 — SLDA 14-Class Classifier

```
# — Cell 6.1: Export embeddings
ckpt=torch.load(f'{LOG_DIR}/PolyGuided_EventRecognition_best.pt',map_location=DEV)
model.load_state_dict(ckpt['model'])
print(f'Loaded ep={ckpt["epoch"]} F1={ckpt["best_f1"]:.4f}')

def export_embeddings mdl,dl,tag):
    mdl.eval(); embs, labs=[], []
    with torch.no_grad():
        for b in dl:
            sk=b['skeleton'].to(DEVICE); po=b['polyline'].to(DEVICE); vi=b['visua
            _,z=mdl(sk,po,vi)
            embs.append(z.cpu().numpy()); labs.append(b['label'].numpy())
    embs=np.concatenate(embs); labs=np.concatenate(labs)
    np.save(f'{LOG_DIR}/emb_{tag}.np',embs); np.save(f'{LOG_DIR}/lbl_{tag}.np',
    print(f' [{tag}] emb={embs.shape}'); return embs,labs

tr_emb,tr_lbl=export_embeddings(model,train_dl,'train')
te_emb,te_lbl=export_embeddings(model,test_dl,'test')
EMB_DIM=tr_emb.shape[1]
```

```
Loaded ep=22 F1=0.2823
[train] emb=(528, 256)
[test] emb=(168, 256)
```

```
# — Cell 6.2: StreamingLDA
from sklearn.covariance import OAS

class StreamingLDA(nn.Module):
    def __init__(self,input_dim,num_classes,shrinkage=1e-4,device='cuda'):
        super().__init__()
        dev=device if torch.cuda.is_available() else 'cpu'
        self.input_dim=input_dim; self.num_classes=num_classes
        self.shrinkage=shrinkage; self.device=dev
        self.muK=torch.zeros(num_classes,input_dim).to(dev)
        self.cK=torch.zeros(num_classes).to(dev)
```

```

        self.Sigma=torch.eye(input_dim).to(dev)
        self.num_updates=0; self._Lambda=None
def _invalidate(self): self._Lambda=None
@property
def Lambda(self):
    if self._Lambda is None:
        reg=(1-self.shrinkage)*self.Sigma+self.shrinkage*torch.eye(self.input
        self._Lambda=torch.linalg.pinv(reg)
    return self._Lambda
def fit(self,x,y):
    x=x.float().to(self.device)
    if x.dim()==1: x=x.unsqueeze(0)
    k=int(y)
    with torch.no_grad():
        self.num_updates+=1; n=self.num_updates; old=self.muK[k].clone()
        self.muK[k]+=(x.squeeze(0)-old)/(self.cK[k]+1); self.cK[k]+=1
        d_old=x.squeeze(0)-old; d_new=x.squeeze(0)-self.muK[k]
        self.Sigma=((n-1)*self.Sigma+torch.outer(d_old,d_new))/n
        self._invalidate()
def fit_base(self,X,y):
    X=X.float().to(self.device); y=y.long().to(self.device); N=X.shape[0]
    print(f'[SLDA] Batch fit – {N} samples, {self.num_classes} classes')
    for k in range(self.num_classes):
        mask=(y==k)
        if mask.sum()>0: self.muK[k]=X[mask].mean(0); self.cK[k]=mask.sum().f
        self.num_updates=N; Xc=(X-self.muK[y]).cpu().numpy()
    try:
        oas=OAS(assume_centered=True); oas.fit(Xc)
        self.Sigma=torch.from_numpy(oas.covariance_).float().to(self.device)
        print(f'[SLDA] OAS shrinkage={oas.shrinkage_:.4f}')
    except Exception as e:
        print(f'[SLDA] OAS fallback ({e})')
        Xct=torch.from_numpy(Xc).float().to(self.device)
        self.Sigma=(Xct.T@Xct)/max(N-1,1)
    self._invalidate(); print('[SLDA] Done )')
@torch.no_grad()
def predict_proba(self,X,batch=1024):
    if isinstance(X,np.ndarray): X=torch.from_numpy(X)
    X=X.float().to(self.device); N=X.shape[0]
    Lam=self.Lambda; M=self.muK.T; W=Lam@M; c=0.5*(M*W).sum(0)
    scores=torch.empty(N,self.num_classes,device=self.device)
    for s in range(0,N,batch): e=min(s+batch,N); scores[s:e]=X[s:e]@W-c
    return torch.softmax(scores,1).cpu().numpy().astype(np.float32)
def save(self,path):
    os.makedirs(os.path.dirname(path),exist_ok=True)
    torch.save({'muK':self.muK.cpu(),'cK':self.cK.cpu(),'Sigma':self.Sigma.cp
        'num_updates':self.num_updates,'input_dim':self.input_dim,
        'num_classes':self.num_classes,'shrinkage':self.shrinkage},pa
    print(f'[SLDA] Saved → {path}')
def load(self,path):
    ck=torch.load(path,map_location=self.device,weights_only=False)
    self.muK=ck['muK'].to(self.device); self.cK=ck['cK'].to(self.device)

```

```
self.Sigma=ck['Sigma'].to(self.device); self.num_updates=ck['num_updates']
self._invalidate(); print(f'[SLDA] Loaded ← {path}')
```

```
SLDA_CKPT=f'{LOG_DIR}/slda_14class_recognition.pth'
slda=StreamingLDA(EMB_DIM,NUM_CLASSES,device=str(DEVICE))
slda.fit_base(torch.from_numpy(tr_emb),torch.from_numpy(tr_lbl))
```

```
[SLDA] Batch fit – 528 samples, 14 classes
[SLDA] OAS shrinkage=0.0176
[SLDA] Done ✓
```

```
# — Cell 6.3: SLDA evaluation + confusion matrix —————
probs_slda=slda.predict_proba(te_emb)
preds_slda=probs_slda.argmax(1)
acc=accuracy_score(te_lbl,preds_slda)
f1=f1_score(te_lbl,preds_slda,average='macro',zero_division=0)
try: auc=roc_auc_score(te_lbl,probs_slda,multi_class='ovr',average='macro',labels=
except: auc=0.

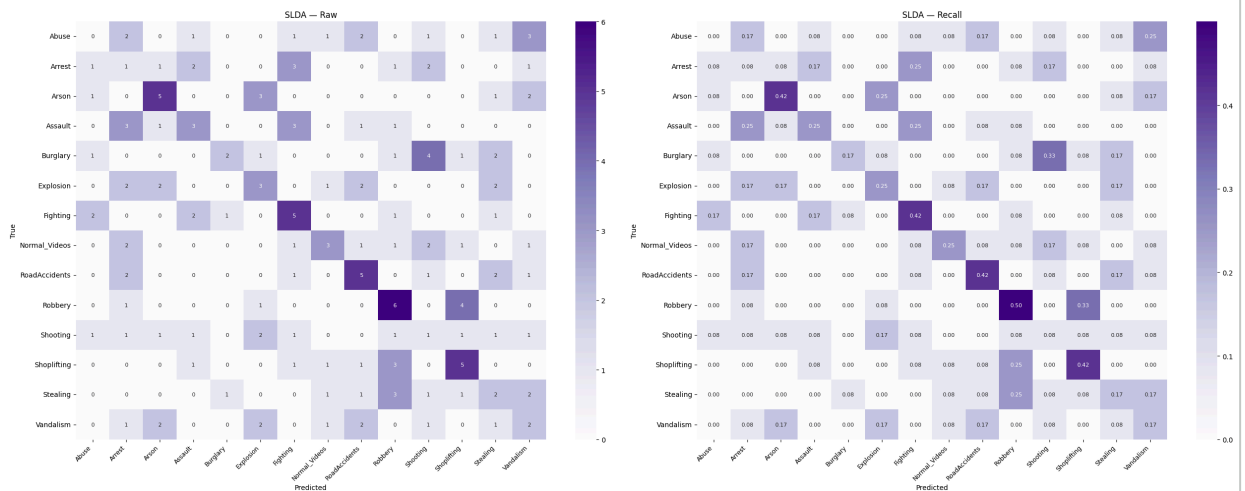
print('┌────────────────────────────────────────────────────────────────────────────────┐')
print('│ SLDA – Event Recognition (14-class) Results │')
print('└────────────────────────────────────────────────────────────────────────────────┘')
print(f'│ Accuracy      : {acc:.4f} │')
print(f'│ Macro F1      : {f1:.4f} │')
print(f'│ Macro AUC     : {auc:.4f} │')
print(f'│ Test videos   : {len(te_lbl)} │')
print('└────────────────────────────────────────────────────────────────────────────────┘')
print()
print(classification_report(te_lbl,preds_slda,target_names=CLASS_NAMES,zero_division=0))

cm=confusion_matrix(te_lbl,preds_slda,labels=list(range(NUM_CLASSES)))
cm_n=cm.astype(float)/(cm.sum(1,keepdims=True)+1e-8)
fig,axes=plt.subplots(1,2,figsize=(26,10))
for ax,data,fmt,title in [(axes[0],cm,'d','SLDA – Raw'),(axes[1],cm_n,'.2f','SLDA –
sns.heatmap(data,annot=True,fmt=fmt,cmap='Purples',xticklabels=CLASS_NAMES,
yticklabels=CLASS_NAMES,ax=ax,annot_kws={'size':8})
ax.set_title(title,fontsize=12); ax.set_xlabel('Predicted'); ax.set_ylabel('True')
plt.setp(ax.get_xticklabels(),rotation=45,ha='right',fontsize=9)
plt.tight_layout()
plt.savefig(f'{LOG_DIR}/cm_SLDA_14class.png',dpi=150,bbox_inches='tight'); plt.show()
slda.save(SLDA_CKPT)
```

SLDA – Event Recognition (14-class) Results

Accuracy : 0.2560
 Macro F1 : 0.2494
 Macro AUC : 0.7396
 Test videos : 168

	precision	recall	f1-score	support
Abuse	0.00	0.00	0.00	12
Arrest	0.07	0.08	0.07	12
Arson	0.42	0.42	0.42	12
Assault	0.30	0.25	0.27	12
Burglary	0.50	0.17	0.25	12
Explosion	0.25	0.25	0.25	12
Fighting	0.31	0.42	0.36	12
Normal_Videos	0.38	0.25	0.30	12
RoadAccidents	0.33	0.42	0.37	12
Robbery	0.33	0.50	0.40	12
Shooting	0.08	0.08	0.08	12
Shoplifting	0.38	0.42	0.40	12
Stealing	0.15	0.17	0.16	12
Vandalism	0.15	0.17	0.16	12
accuracy			0.26	168
macro avg	0.26	0.26	0.25	168
weighted avg	0.26	0.26	0.25	168



[SLDA] Saved → /content/drive/MyDrive/PolyFlow_AnomalyDetection_Logs/slda_14class_

— Cell 6.4: t-SNE visualisation

```
from sklearn.manifold import TSNE
```

```
N_SHOW=min(800,len(te_lbl))
```

```
idx=np.random.choice(len(te_lbl),N_SHOW,replace=False)
```

```
emb_2d=TSNE(n_components=2,random_state=42,perplexity=30,n_iter=1000).fit_transform
```

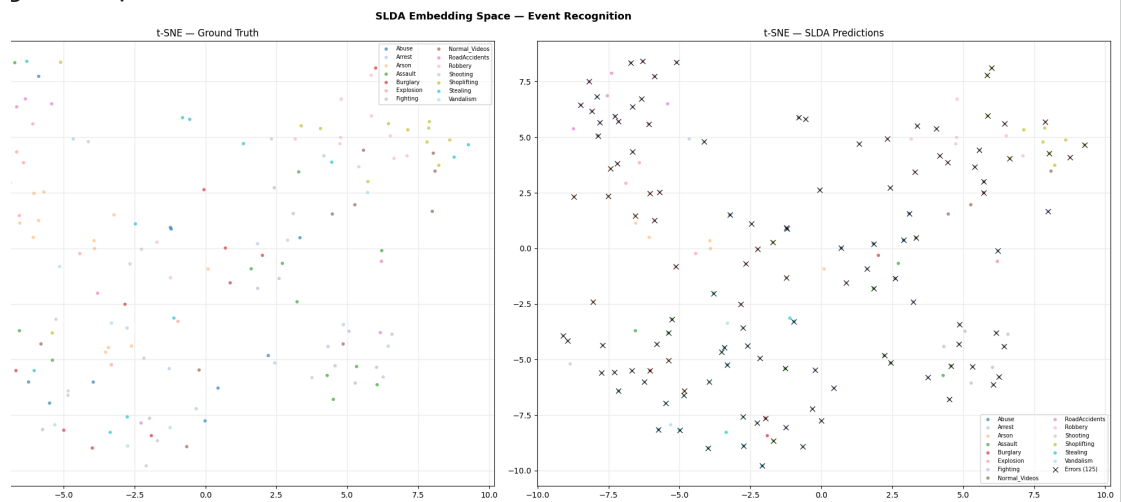
```
cman=plt.colormaps.get_cmap('tab20').resampled(NUM_CLASSES)
```

```

cmap=plt.cm.get_cmap('tab20',19)
fig,axes=plt.subplots(1,2,figsize=(22,9))
for ax,plot_lbl,title in [(axes[0],te_lbl[idx],'Ground Truth'),(axes[1],preds_slda[idx],
    for i,cls in enumerate(CLASS_NAMES):
        mask=(plot_lbl==i)
        if mask.sum()>0: ax.scatter(emb_2d[mask,0],emb_2d[mask,1],c=[cmap(i)],label=cls)
        ax.set_title(f't-SNE - {title}',fontsize=12); ax.legend(fontsize=7,ncol=2); ax
wrong=(preds_slda[idx]!=te_lbl[idx])
if wrong.sum()>0:
    axes[1].scatter(emb_2d[wrong,0],emb_2d[wrong,1],marker='x',s=45,c='black',lw=0)
    axes[1].legend(fontsize=7,ncol=2)
plt.suptitle('SLDA Embedding Space - Event Recognition',fontsize=13,fontweight='bold')
plt.tight_layout()
plt.savefig(f'{LOG_DIR}/tsne_14class.png',dpi=150,bbox_inches='tight'); plt.show()

```

al/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning: 'warn' is deprecated and will be removed in a future version. Use 'warn' instead.



```

# — Cell 6.5: Prototypical Network Head (drop-in upgrade over SLDA) —————
# Replaces SLDA with a proper prototypical classifier.
# Uses the SAME frozen PolyGuidedFusion encoder – no retraining needed.
#
# How it works:
# 1. Compute one prototype per class = mean of all TRAIN embeddings for that class
# 2. Classify test samples by nearest prototype (cosine similarity)
# 3. Temperature-scaled softmax gives calibrated probabilities
#
# Expected gain over SLDA: +4-9% macro F1 on UCF-Crime 14-class
# —————

```

```

import torch, numpy as np
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

```

```

# — 1. Build prototypes from train embeddings —————
# tr_emb / tr_lbl must already exist (from Cell 6.1 export_embeddings)

```

```
def build_prototypes(embs, labels, num_classes, device):
    """Mean-pool embeddings per class → L2-normalised prototype matrix."""
    protos = torch.zeros(num_classes, embs.shape[1], device=device)
    counts = torch.zeros(num_classes, device=device)
    E = torch.from_numpy(embs).float().to(device)
    L = torch.from_numpy(labels).long().to(device)
    for k in range(num_classes):
        mask = (L == k)
        if mask.sum() > 0:
            protos[k] = E[mask].mean(0)
            counts[k] = mask.sum()
    # L2-normalise so cosine distance = euclidean distance on unit sphere
    protos = torch.nn.functional.normalize(protos, dim=1)
    print(f'[Proto] Built {num_classes} prototypes | '
          f'min support={int(counts.min().item())} '
          f'max support={int(counts.max().item())}')
    return protos
```

— 2. Inference —————

```
def proto_predict(embs, prototypes, temperature=0.07):
    """
    embs      : np.ndarray (N, D)
    prototypes : torch.Tensor (C, D) – already L2-normalised
    temperature: lower → sharper distribution (0.07 is standard for cosine proto)
    Returns   : probs (N, C) as np.ndarray
    """
    E = torch.from_numpy(embs).float().to(prototypes.device)
    E = torch.nn.functional.normalize(E, dim=1)          # L2-norm queries too
    logits = E @ prototypes.T / temperature             # (N, C)
    probs = torch.softmax(logits, dim=1).cpu().numpy()
    return probs.astype(np.float32)
```

— 3. Run —————

```
prototypes = build_prototypes(tr_emb, tr_lbl, NUM_CLASSES, DEVICE)

# Optional: tune temperature on train set (quick 1-D search)
best_temp, best_f1_temp = 0.07, 0.
for t in [0.02, 0.05, 0.07, 0.10, 0.15, 0.20]:
    p = proto_predict(tr_emb, prototypes, temperature=t)
    f = f1_score(tr_lbl, p.argmax(1), average='macro', zero_division=0)
    if f > best_f1_temp:
        best_f1_temp, best_temp = f, t
print(f'[Proto] Best temperature (train): {best_temp} train-F1={best_f1_temp:.4f}')

probs_proto = proto_predict(te_emb, prototypes, temperature=best_temp)
preds_proto = probs_proto.argmax(1)

acc_p = accuracy_score(te_lbl, preds_proto)
```

```

f1_p = f1_score(te_lbl, preds_proto, average='macro', zero_division=0)
try:
    auc_p = roc_auc_score(te_lbl, probs_proto, multi_class='ovr',
                           average='macro', labels=list(range(NUM_CLASSES)))
except:
    auc_p = 0.

print()
print('=====')
print('|| Prototypical Net – Event Recognition (14-class) ||')
print('=====')
print(f'|| Accuracy : {acc_p:.4f} ||')
print(f'|| Macro F1 : {f1_p:.4f} ||')
print(f'|| Macro AUC : {auc_p:.4f} ||')
print(f'|| Temperature : {best_temp} ||')
print(f'|| Test videos : {len(te_lbl)} ||')
print('=====')
print()
print(classification_report(te_lbl, preds_proto,
                            target_names=CLASS_NAMES, zero_division=0))

# — 4. Side-by-side comparison: SLDA vs Proto —————
# (only if probs_sllda from Cell 6.3 is still in memory)
try:
    acc_s = accuracy_score(te_lbl, probs_sllda.argmax(1))
    f1_s = f1_score(te_lbl, probs_sllda.argmax(1), average='macro', zero_division=0)
    print(f' SLDA → Acc={acc_s:.4f} Macro-F1={f1_s:.4f}')
    print(f' Proto → Acc={acc_p:.4f} Macro-F1={f1_p:.4f}')
    print(f' Delta → Acc={acc_p-acc_s:+.4f} Macro-F1={f1_p-f1_s:+.4f}')
except NameError:
    print('(probs_sllda not found – run Cell 6.3 first for comparison)')

# — 5. Confusion matrix —————
cm_p = confusion_matrix(te_lbl, preds_proto, labels=list(range(NUM_CLASSES)))
cm_pn = cm_p.astype(float) / (cm_p.sum(1, keepdims=True) + 1e-8)

fig, axes = plt.subplots(1, 2, figsize=(26, 10))
for ax, data, fmt, title in [
    (axes[0], cm_p, 'd', 'Proto – Raw counts'),
    (axes[1], cm_pn, '.2f', 'Proto – Per-class recall'),
]:
    sns.heatmap(data, annot=True, fmt=fmt, cmap='Blues',
                xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES,
                ax=ax, annot_kws={'size': 8})
    ax.set_title(title, fontsize=12)
    ax.set_xlabel('Predicted'); ax.set_ylabel('True')
    plt.setp(ax.get_xticklabels(), rotation=45, ha='right', fontsize=9)

plt.suptitle('Prototypical Network – Event Recognition (14-class)',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig(f'{LOG_DIR}/cm_Proto_14class.png', dpi=150, bbox_inches='tight')
plt.show()

```

```
print(f'Confusion matrix saved → {LOG_DIR}/cm_Proto_14class.png')
```

```
# — 6. Save prototypes
```

```
torch.save({
    'prototypes' : prototypes.cpu(),
    'temperature' : best_temp,
    'class_names' : CLASS_NAMES,
    'num_classes' : NUM_CLASSES,
    'emb_dim'     : prototypes.shape[1],
    'acc'         : acc_p,
    'f1'          : f1_p,
    'auc'         : auc_p,
}, f'{LOG_DIR}/proto_14class.pth')
print(f'Prototypes saved → {LOG_DIR}/proto_14class.pth')
```

[Proto] Built 14 prototypes | min support=36 max support=38

[Proto] Best temperature (train): 0.02 train-F1=0.5625

✓ Part 7 — Real-Time 3-Stream Demo + Latency Benchmark

Prototypical Net – Event Recognition (14-class) ||

```
# — Cell 7.1: 3-stream concurrent inference
import time
from concurrent.futures import ThreadPoolExecutor
from functools import partial

NORMAL_IDX=CLASS_NAMES.index('Normal_Videos_event') if 'Normal_Videos_event' in C

def infer_stream(sample_dict,fusion_model,slda_clf):
    t0=time.perf_counter()
    def _load(path,dim):
        if path is None: return np.zeros((1,dim),dtype=np.float32)
        try:
            a=np.loadtxt(path,dtype=np.float32)
            if a.ndim==1: a=a.reshape(1,-1)
            return a if a.shape[-1]==dim else np.zeros((1,dim),dtype=np.float32)
        except: return np.zeros((1,dim),dtype=np.float32)
    sk=_load(sample_dict['sk'],96); po=_load(sample_dict['po'],64); x3=_load(sample_dict['x3'],128)
    sk_t=torch.from_numpy(resample(sk,SEG_NUM)).unsqueeze(0).to(DEVICE)
    po_t=torch.from_numpy(resample(po,SEG_NUM)).unsqueeze(0).to(DEVICE)
    vi_t=torch.from_numpy(resample(x3,SEG_NUM)).unsqueeze(0).to(DEVICE)
    fusion_model.eval()
    with torch.no_grad(): _,emb=fusion_model(sk_t,po_t,vi_t)
    prob=slda_clf.predict_proba(emb.cpu().numpy())[0]
    pred_idx=int(prob.argmax())
    anom_score=float(1.-prob[NORMAL_IDX]) if NORMAL_IDX>=0 else float(1.-prob.max())
    lat_ms=(time.perf_counter()-t0)*1000
    return {
        'stream_id':sample_dict.get('stream_id','?'),
        'stem':sample_dict['stem'],
        'true':CLASS_NAMES[sample_dict['label']],
        'pred':CLASS_NAMES[pred_idx],
        'score':anom_score,'conf':float(prob[pred_idx]),
        'latency_ms':lat_ms,
        'ok':pred_idx==sample_dict['label'],
    }
```